

從 LightNobel 到 FPGA 記憶體優化教學手冊

蛋白質結構預測模型的 memory-bound 問題，以及在 AMD Alveo U55C 上的常見解法

專題 reference : LightNobel (ISCA 2025)

程式碼 : github.com/antony3911/ForClaude (分支 `claude/lightnobel-memory-optimization-yhokyc`)

產生日期 : 2026-09-26

目錄

第 1 章 為何而做：問題具體發生了什麼？

- 1.1 蛋白質結構預測在做什麼？
- 1.2 AlphaFold2 / ESMFold 的架構全貌
- 1.3 Triangle Multiplication 在做什麼？
- 1.4 問題一：記憶體放不下
- 1.5 問題二：搬資料比計算慢 (memory-bound)
- 1.6 LightNobel 的觀察與解法 (高層次)
- 1.7 我們的定位
- 本章小結

第 2 章 為何可做：這些解法從什麼概念上解決問題？

- 2.1 一條公式看懂所有優化
- 2.2 記憶體階層：為什麼「搬運」有成本
- 2.3 主軸概念：算術強度與 Roofline 模型
- 2.4 解法一：Tiling／資料重用 → 減少搬運總量 ($v0 \rightarrow v1$)
- 2.5 解法二：寬位元與 burst → 提高每次搬運的效率 ($v1 \rightarrow v2$)
- 2.6 解法三：Double buffering → 把等待時間藏起來 ($v2 \rightarrow v3$)
- 2.7 解法四：多通道 HBM → 多條路同時搬 (cfg 對照實驗)
- 2.8 解法五：量化 → 讓每筆資料變小 ($v2 \rightarrow v4$)
- 2.9 其他常見手法 (本專題未實作，但報告中可以提)
- 2.10 為什麼「效益可能不大」仍然算解法？
- 本章小結

第 3 章 整體流程：從頭到現在具體做了什麼？

- 3.1 全景圖
- 3.2 步驟一：界定範圍
- 3.3 步驟二：資料排列 (memory layout) 與算式
- 3.4 步驟三：寫 5 個 kernel
- 3.5 步驟四：正確性驗證 (C 模擬)
- 3.6 步驟五：host 程式
- 3.7 步驟六：HBM 連接設定
- 3.8 步驟七：Makefile
- 3.9 步驟八：真實資料管線 (加分項)
- 3.10 步驟九：在實際 server 上除錯
- 3.11 步驟十：讀懂 HLS 報告 (以 v2 為例)
- 3.12 所有檔案總表
- 本章小結

第 4 章 工具原理：為何 C++ 可以變成電路？

- 4.1 FPGA 是什麼？
- 4.2 為什麼 FPGA 適合做記憶體優化的實驗？
- 4.3 HLS：C++ 是怎麼變成電路的？
- 4.4 為什麼 C++ 可以直接模擬？
- 4.5 Vivado：從 RTL 到 bitstream
- 4.6 Vitis 與 v++：把 kernel 接到板子上
- 4.7 XRT：host 程式怎麼控制 FPGA
- 4.8 Python 端：PyTorch、HuggingFace、ESMFold
- 4.9 誰提供了什麼能力？
- 4.10 Claude 在這個對話中呼叫的工具
- 本章小結

第 5 章 研究方法：之後可以重複使用的步驟

- 5.1 整體流程
- 5.2 步驟一：讀 paper 的方法
- 5.3 步驟二：量化問題
- 5.4 步驟三：縮小範圍
- 5.5 步驟四：設計實驗
- 5.6 步驟五：實作與驗證
- 5.7 步驟六：分析
- 5.8 步驟七：呈現（報告要畫的圖）
- 5.9 除錯的思路（以 2025.2 路徑 bug 為例）
- 5.10 本專題套用這個流程的完整範例
- 本章小結

第 6 章 術語表

- 6.1 蛋白質與模型
- 6.2 效能與記憶體概念
- 6.3 量化
- 6.4 FPGA 硬體
- 6.5 HLS 與硬體設計
- 6.6 AMD 工具鏈
- 6.7 軟體與開發工具
- 本章小結

第 7 章 指令手冊：做過的、可以做的、格式與用途

- 7.1 先看懂指令的格式
- 7.2 Linux 基本操作
- 7.3 環境調查與設定
- 7.4 Git：取得與更新程式碼
- 7.5 tmux：斷線也不會中斷的終端機
- 7.6 本專案的 make 指令（最重要）

7.7 Vitis 工具的原始指令 (Makefile 背後在做什麼)

7.8 看結果

7.9 host 程式與 Python 腳本的參數

7.10 目前為止實際執行過的指令 (時間軸)

7.11 常見錯誤與排查

本章小結

第 1 章 為何而做：問題具體發生了什麼？

本章重點

- 蛋白質結構預測模型（AlphaFold2、ESMFold）內部有一份 $L \times L \times 128$ 的巨大資料（pair representation）。
- 蛋白質越長（ L 越大），這份資料以平方成長：長度加倍，資料變 4 倍。
- 這造成兩個具體問題：① 記憶體放不下（序列長度受限），② 搬資料比計算還慢（memory-bound）。
- LightNobel 針對這兩個問題提出解法；我們的專題則是在 FPGA 上，用常見手法處理其中最關鍵的一個運算。

1.1 蛋白質結構預測在做什麼？

白話直覺

蛋白質是一條由胺基酸串成的「珠鍊」。胺基酸共有 20 種，用英文字母表示，所以一條蛋白質可以寫成一串字：

```
MKTAYIAKQRQISFVKSHFSRQLEERLGLIEVQAPILSRVGDGTQDNLSGAEK...
```

這條珠鍊在細胞裡會自動摺疊成特定的 3D 形狀，而形狀決定功能：酵素能不能抓住受質、抗體能不能卡住病毒，都取決於形狀。

蛋白質結構預測 = 輸入那串字母，輸出每個原子的 3D 座標。

為什麼重要

- 用實驗測定結構（X 光繞射、冷凍電顯）要花數月到數年，而且很貴。
- AlphaFold2（DeepMind，2021）讓預測準確度接近實驗水準，相關研究獲得 2024 年諾貝爾化學獎。LightNobel 的名字就是在呼應這件事。
- 應用：藥物設計、理解疾病突變、設計新的酵素。

本專題的關鍵變數：L

L = 蛋白質的長度（胺基酸個數）。整份手冊都圍繞這個數字：

蛋白質	UniProt ID	L
血紅素 α 鏈	P69905	142
p53（抑癌蛋白）	P04637	393
新冠病毒棘蛋白（單體）	P0DTC2	1,273

蛋白質	UniProt ID	L
BRCA1 (乳癌相關)	P38398	1,863
Dystrophin (肌肉蛋白)	P11532	3,685
Titin (人體最大的蛋白之一)	Q8WZ42	約 34,000

大部分蛋白質在 1,000 以內，但很多重要的蛋白質比這長。另外，預測蛋白質複合體時常把多條鏈接起來一起算，L 會變得更大。

1.2 AlphaFold2 / ESMFold 的架構全貌

我們的 reference (LightNobel) 以這一類模型為對象，統稱 PPM (Protein structure Prediction Model)。我們實際對照的是 ESMFold (Meta, 2022)，因為它的結構比 AlphaFold2 簡單，而且公開在 HuggingFace 上。



(AlphaFold2 的對應部分叫 Evoformer，也有 48 個 block，同樣有 pair representation。差別在於 AlphaFold2 用多序列比對 (MSA) 當輸入，ESMFold 改用語言模型。)

兩種 representation

名稱	形狀	意義	大小隨 L
Sequence representation	$L \times 1024$	「第 i 個胺基酸」的特徵	線性 (L)
Pair representation	$L \times L \times 128$	「第 i 個和第 j 個胺基酸之間的關係」	平方 (L^2)

Pair representation 可以想成一張 $L \times L$ 的表格，每一格放 128 個數字。第 (i, j) 格記錄模型對「i 和 j 距離多遠、有沒有交互作用」的理解。

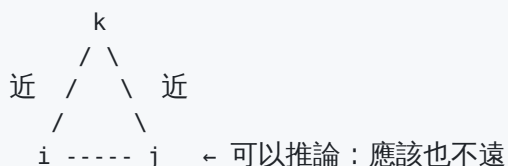
1.3 Triangle Multiplication 在做什麼？

「三角形」的直覺

如果知道：

- 胺基酸 i 離 k 很近
- 胺基酸 j 離 k 也很近

那 i 和 j 大概也不會太遠，這就是三角不等式。



Triangle Multiplication 就是把這個推理寫成數學：更新 (i, j) 格時，把所有第三者 k 都看過一遍。

算式 (outgoing 版本)

$$z[i][j][c] = \sum_k a[i][k][c] \times b[j][k][c] \quad (k \text{ 從 } 0 \text{ 到 } L-1, c \text{ 從 } 0 \text{ 到 } 127)$$

- a、b：由 pair representation 經過 LayerNorm、線性轉換、gating 得到，形狀都是 $L \times L \times 128$ 。
- z：輸出，形狀也是 $L \times L \times 128$ 。
- 對每個 channel c 來說，這其實是一個 $L \times L$ 的矩陣乘法，總共做 128 次。

具體例子 (L = 3, 只看一個 channel)

$$a = \begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \\ a_{20} & a_{21} & a_{22} \end{bmatrix} \quad b = \begin{bmatrix} b_{00} & b_{01} & b_{02} \\ b_{10} & b_{11} & b_{12} \\ b_{20} & b_{21} & b_{22} \end{bmatrix}$$

$$\begin{aligned} z[0][1] &= a_{00} \cdot b_{10} + a_{01} \cdot b_{11} + a_{02} \cdot b_{12} \\ &= \sum_k a[0][k] \cdot b[1][k] \quad \leftarrow \text{用到 } a \text{ 的第 } 0 \text{ 列和 } b \text{ 的第 } 1 \text{ 列} \end{aligned}$$

也就是 $z = a \times b^T$ (a 乘以 b 的轉置)。

我們已經用 HuggingFace 的 ESMFold 驗證：這個算式和 ESMFold 模組內部的計算完全一致（差值 0）（詳見第 3 章 3.9 節）。

Triangle Attention（我們沒有實作，但 LightNobel 有處理）

同一個 block 裡還有 Triangle Attention：對每一列 i ，在 (j, k) 之間做 attention。它的中間值（attention logits）形狀是 $L \times L \times L \times \text{heads}$ ，是立方成長，所以實作上一定要分塊（chunking）計算，否則 $L = 1000$ 時光這個中間值就需要數十 GB。

1.4 問題一：記憶體放不下

算一算 pair representation 有多大

每個數字用 FP32（4 bytes），每格 128 個數字 → 每格 512 bytes。

$$\text{大小} = L \times L \times 128 \times 4 \text{ bytes} = L^2 \times 512 \text{ bytes}$$

L	L^2	一份 pair tensor (FP32)
256	65,536	32 MB
500	250,000	128 MB
1,000	1,000,000	512 MB
2,000	4,000,000	2 GB
4,000	16,000,000	8 GB
10,000	100,000,000	51 GB

長度變 2 倍，資料變 4 倍；長度變 10 倍，資料變 100 倍。

實際需要的不只一份

計算一次 Triangle Multiplication 時，同時存在的資料至少有：

資料	形狀	說明
輸入 z	$L \times L \times 128$	pair representation 本身
LayerNorm 後的 z	$L \times L \times 128$	
a 、 b	各 $L \times L \times 128$	投影後的兩份
gate (a_g 、 b_g 、 g)	各 $L \times L \times 128$	gating 用
輸出	$L \times L \times 128$	

粗估同時有 5~8 份 pair 大小的資料。L = 2000 時，光這一步就可能需要 10~16 GB。再加上 Triangle Attention 的立方級中間值，GPU 記憶體很快就不夠用（Out of Memory, OOM）。

為什麼問題出在 activation，不是模型權重

類別	是什麼	大小和 L 的關係	ESMFold 的量級
權重 (weights)	模型學到的參數	固定，和 L 無關	約 3~4B 參數，FP16 約 7~8 GB
Activation	計算過程中產生的中間資料	隨 L^2 甚至 L^3 成長	L 大時可達數十 GB

L 小的時候，權重佔大頭；L 一大，activation 就遠遠超過權重。所以「把模型變小」幫助有限，真正要處理的是 activation。這正是 LightNobel 的切入點。

現在的解法與代價

常見做法是 chunking（分塊）：一次只算一部分，算完就丟。

- 優點：記憶體峰值下降。
- 代價：要重複讀資料、無法完全平行，時間變長。

小結：序列長度被記憶體容量卡住，這就是 LightNobel 標題裡的「Sequence Length Limitation」。

1.5 問題二：搬資料比計算慢（memory-bound）

白話直覺：廚房比喻

- HBM / DRAM（主記憶體）= 遠處的倉庫：很大，但來回一趟很花時間。
- 晶片內記憶體（cache、BRAM、shared memory）= 手邊的料理台：拿取很快，但很小。
- 運算單元 = 廚師：切菜很快。

如果廚師切一刀，就要等助手跑一趟倉庫拿下一個食材，那廚師大部分時間都在等。這時候瓶頸是「搬運」，不是「廚師的速度」。這就叫 memory-bound。反過來，如果食材都在手邊，只是廚師切不完，就叫 compute-bound。

用數字判斷：算術強度（Arithmetic Intensity）

算術強度 = 運算次數（FLOP） ÷ 從主記憶體搬運的資料量（byte）

單位是 FLOP/byte，代表「每搬 1 byte 可以做幾次運算」。越高越好。

例子：最直白的寫法（我們的 v0）

每算一次 $a \times b$ 並累加（2 次運算：一次乘、一次加），都要從記憶體讀一個 a （4 bytes）和一個 b （4 bytes）：

$$\text{算術強度} = 2 \text{ FLOP} \div 8 \text{ bytes} = 0.25 \text{ FLOP/byte}$$

硬體能提供多少？

硬體	記憶體頻寬	峰值運算能力 (FP32)	平衡點 (運算 ÷ 頻寬)
U55C (本專題)	約 460 GB/s (HBM2)	取決於設計	—
一般 GPU (例：A100)	約 1.5~2 TB/s	約 19.5 TFLOP/s	約 10 FLOP/byte

以 GPU 為例，演算法的算術強度要超過約 10 FLOP/byte，才能把運算單元餵飽。v0 只有 0.25，也就是說運算單元大約 97% 的時間都在等資料。

那 Triangle Multiplication 本質上是 memory-bound 嗎？

不一定，取決於怎麼寫：

- 理論上：總運算量是 $2 \cdot L^3 \cdot C$ ，但資料只有 $3 \cdot L^2 \cdot C$ 個數字。如果每個數字只搬一次，算術強度可以非常高 (L 越大越高)。
- 實際上：晶片內的空間放不下整個 $L \times L \times 128$ ，只能分批搬，而分批的方式決定要重複搬幾次。

這就是優化的空間所在：同樣的計算，搬運方式不同，速度可以差幾十倍。第 2 章會用公式說明每種手法如何減少搬運；第 3 章會看到實際數字 (v2 報告中約 21% 的時間在等記憶體)。

1.6 LightNobel 的觀察與解法 (高層次)

以下是概念層次的整理。具體數字、實驗設定與硬體細節請以原文為準。

他們的觀察

1. PPM 的記憶體瓶頸主要來自 activation (尤其是 pair representation)，而不是權重。
2. Activation 的數值分布在不同 token 之間差異很大。少數 token 有特別大的數值 (outlier)。(這裡的 token 指 pair representation 中的一格，也就是一個 (i, j) 位置的 128 個數字。)
3. 如果整份資料共用同一個量化尺度，outlier 會讓其他數值的精度嚴重下降。

他們的解法

1. Token-wise Adaptive Activation Quantization (AAQ)：每個 token 各自決定量化方式 (包含尺度，以及如何處理 outlier)，在大幅壓縮 activation 的同時維持預測準確度。
2. 專用硬體加速器：設計能有效率地處理這種「每個 token 格式不同」的資料的運算單元 (原文中有可重組的矩陣運算單元與向量運算單元等設計)。
3. 結果：大幅降低記憶體峰值，讓模型能處理比 GPU 基準長得多的序列，同時保持速度。

和我們的關係

LightNobel	我們的專題
整個 PPM 的完整解法	只取 Triangle Multiplication 的核心 einsum
自己設計 ASIC 加速器（模擬評估）	用現成的 FPGA（U55C）實際實作
提出新的量化方法（AAQ）	實作簡化版：INT8 per-token（v4），並與 per-tensor 比較
新穎的架構設計	比較常見的 memory-bound 手法（tiling、寬位元、double buffering、多通道、量化）

我們在模擬資料上已經重現了 LightNobel 的核心直覺：資料中有 1% 的 outlier token 時，per-token INT8 的誤差約 1%，per-tensor INT8 約 11%，相差約 10 倍。

1.7 我們的定位

題目一句話

在 FPGA 上實作 PPM 中最吃記憶體體的運算（Triangle Multiplication），逐一套用常見的 memory-bound 優化手法，量化每一招帶來多少改善、付出多少硬體代價。

為什麼選 Triangle Multiplication

1. 它是 pair representation 的核心運算，直接受 L^2 資料量影響。
2. 算式簡單（本質是矩陣乘法），容易驗證正確性。
3. 可以清楚展示「同樣的計算，不同搬運方式」的差異。
4. 量化的效果（per-token vs per-tensor）可以直接呼應 LightNobel。

為什麼用 FPGA

- FPGA 可以自己決定記憶體怎麼接、資料怎麼搬、運算單元擺幾個，適合展示記憶體優化的每一個細節。
- U55C 有 HBM（多通道高頻寬記憶體），可以做多通道的實驗。
- GPU 上這些細節大多被硬體和驅動程式隱藏，比較難單獨觀察每一招的效果。

本章小結

問題	具體表現	例子
記憶體容量	pair representation 隨 L^2 成長	$L=2000$ 時一份就 2 GB，同時存在 5~8 份
記憶體頻寬	搬資料比計算慢	最直白的寫法算術強度只有 0.25 FLOP/byte

問題	具體表現	例子
精度 vs. 壓縮	outlier 讓量化誤差變大	per-tensor INT8 誤差是 per-token 的約 10 倍

下一章會解釋：為什麼 tiling、寬位元、double buffering、多通道、量化這些手法，可以從概念上解決（或緩解）這些問題。

第 2 章 為何可做：這些解法從什麼概念上解決問題？

本章重點

- 所有記憶體優化都可以用一條「時間公式」來理解：每一招都在縮小公式裡的某一項。
- 算術強度與 Roofline 模型用來判斷瓶頸在哪，也用來評估優化有沒有效。
- 五個手法各自對應的概念：Tiling → 減少搬運總量；寬位元／burst → 提高每次搬運的效率；Double buffering → 把等待時間藏起來；多通道 → 多條路同時搬；量化 → 每筆資料變小。
- 「效益不大」通常是因為瓶頸轉移了（Amdahl's Law），這本身就是有價值的發現。

2.1 一條公式看懂所有優化

一個 kernel 的執行時間，可以粗略拆成「計算」和「搬資料」兩部分：

如果計算和搬運「輪流」做（沒有重疊）：

$$T \approx T_{\text{compute}} + T_{\text{memory}}$$

如果計算和搬運「同時」做（完全重疊）：

$$T \approx \max(T_{\text{compute}}, T_{\text{memory}})$$

其中：

$$T_{\text{compute}} = \text{運算次數} \div (\text{每個 cycle 的運算數} \times \text{時脈})$$

$$T_{\text{memory}} = \text{搬運量} \div \text{頻寬} + \text{每次請求的延遲} \times \text{等待次數}$$

每一招優化，都是在縮小其中一項：

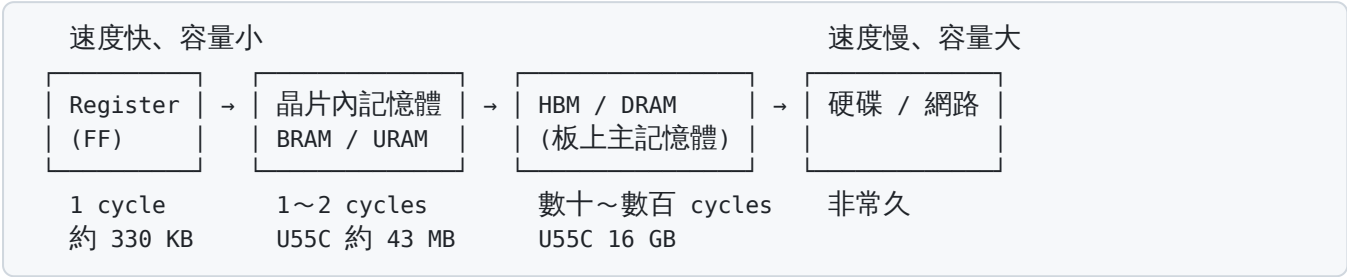
手法	縮小哪一項	對應版本
Tiling／資料重用	搬運量 ↓	v1
寬位元、burst	頻寬 ↑、等待次數 ↓	v2
平行運算單元	每 cycle 運算數 ↑	v2 (16 路)、v4 (64 路)
Double buffering	把 + 變成 max	v3
多通道 HBM	頻寬 ↑	cfg 對照實驗
量化	搬運量 ↓ (每筆資料變小)	v4

接下來每一節都會回到這條公式。

2.2 記憶體階層：為什麼「搬運」有成本

白話直覺

離運算單元越近的記憶體，越快但越小；越遠的記憶體，越大但越慢。



U55C 的具體數字

層級	容量	存取延遲	備註
FF (暫存器)	約 260 萬個 bit (約 330 KB)	1 cycle	直接接在運算單元上
BRAM	4,032 塊 × 18 Kb ≈ 9 MB	1 ~ 2 cycles	可同時存取很多塊
URAM	960 塊 × 288 Kb ≈ 35 MB	1 ~ 2 cycles	比 BRAM 大塊
HBM2	16 GB (32 個 pseudo-channel × 512 MB)	數十~上百 cycles	總頻寬約 460 GB/s

關鍵：一份 $L = 1000$ 的 pair tensor 就有 512 MB，晶片內只有約 43 MB，不可能整份放進來，一定得分批從 HBM 搬。

2.3 主軸概念：算術強度與 Roofline 模型

算術強度 (Arithmetic Intensity, AI)

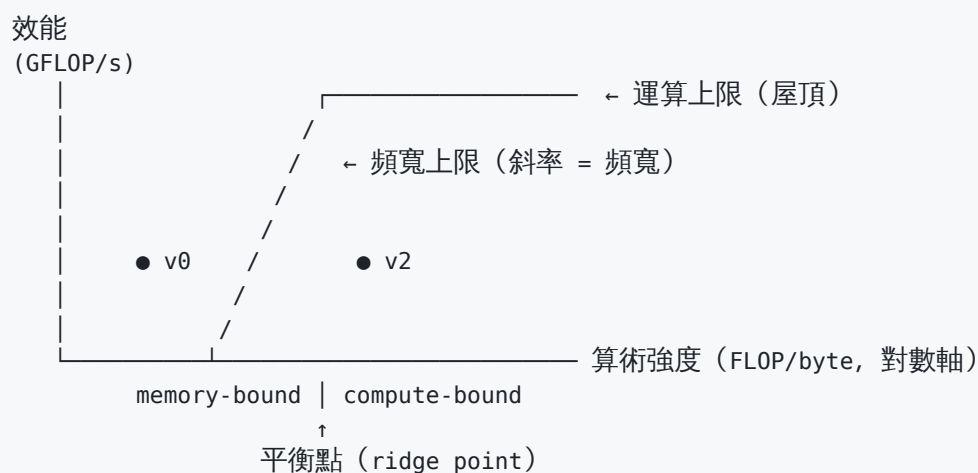
AI = 運算次數 ÷ 從主記憶體搬運的 byte 數 (單位：FLOP/byte)

意義：每搬 1 byte，能做幾次運算。

Roofline 模型

硬體有兩個上限：

- 1. 運算上限 (每秒最多做幾次運算)：一條水平線。
- 2. 頻寬上限 (每秒最多搬多少 byte)：AI 越高，同樣頻寬能支撐的運算越多，是一條斜線。



- 在平衡點左邊：被頻寬限制 (memory-bound) → 要提高 AI 或頻寬。
- 在平衡點右邊：被運算能力限制 (compute-bound) → 要增加運算單元。

可達效能的公式

可達效能 = $\min(\text{運算上限}, \text{AI} \times \text{頻寬})$

本專題的例子

版本	AI (FLOP/byte)	說明
v0	0.25	每 2 次運算讀 8 bytes
v1、v2、v3 (8x8 tile)	2	見 2.4 節推導
v4 (INT8, 8x8 tile)	約 7.8	見 2.8 節推導

畫 roofline 圖是報告中最重要圖之一：可以一眼看出每一招讓 kernel 往哪個方向移動。

2.4 解法一：Tiling／資料重用 → 減少搬運總量 (v0 → v1)

白話直覺

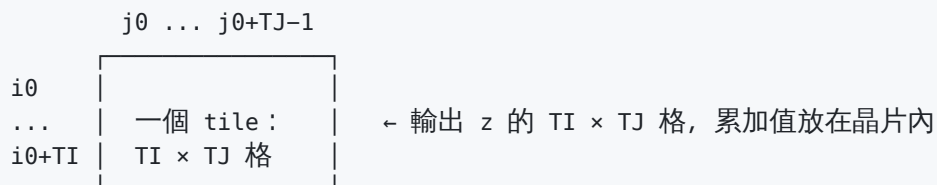
v0 每用一次食材就跑一趟倉庫。v1 一次搬一批到料理台，把這批能做的菜都做完，再搬下一批。

為什麼能重用？

看算式 $z[i][j][c] = \sum_k a[i][k][c] \times b[j][k][c]$ ：

- $a[i][k][c]$ 會被所有的 j 用到 (L 次)。
- $b[j][k][c]$ 會被所有的 i 用到 (L 次)。

如果一次處理 T_I 個 i 和 T_J 個 j (一個 tile)，那搬進來的每個 a 可以用 T_J 次，每個 b 可以用 T_I 次。



對每個 k :

- 搬 $a[i0..i0+TI-1][k][:]$ ($TI \times C$ 個數)
- 搬 $b[j0..j0+TJ-1][k][:]$ ($TJ \times C$ 個數)
- 做 $TI \times TJ \times C$ 次乘加 ← 每個 a 用 TJ 次、每個 b 用 TI 次

公式推導：總搬運量

$v0$ (沒有重用) : 每次乘加讀一個 a 和一個 b 。

讀取次數 = $2 \times L^3 \times C$ (每個 $z[i][j][c]$ 要 L 次乘加)

$v1$ (tiling) :

tile 的數量 = $(L/TI) \times (L/TJ)$
 每個 tile、每個 k 的讀取 = $(TI + TJ) \times C$
 k 的數量 = L

讀取次數 = $(L/TI) \times (L/TJ) \times L \times (TI + TJ) \times C$
 $= L^3 \times C \times (1/TI + 1/TJ)$

比值：

$v0 / v1 = 2 \div (1/TI + 1/TJ)$

$TI = TJ = 8 \rightarrow 2 \div (1/8 + 1/8) = 8$ 倍
 $TI = TJ = 16 \rightarrow 2 \div (1/16 + 1/16) = 16$ 倍
 $TI = TJ = 32 \rightarrow 32$ 倍

算術強度：

$AI = 2 \cdot L^3 \cdot C \div (4 \text{ bytes} \times L^3 \cdot C \cdot (1/TI + 1/TJ))$
 $= TI \cdot TJ \div (2 \cdot (TI + TJ))$

$8 \times 8 \rightarrow 64 \div 32 = 2$ FLOP/byte
 $16 \times 16 \rightarrow 256 \div 64 = 4$ FLOP/byte
 $32 \times 32 \rightarrow 1024 \div 128 = 8$ FLOP/byte

代價：晶片內記憶體

累加器要放下整個 tile 的輸出： $TI \times TJ \times C$ 個 float。| Tile | 累加器大小 | 佔晶片內記憶體 | |---|---|---| |
 8×8 | 8,192 個 float = 32 KB | 很少 | | 16×16 | 32,768 個 = 128 KB | 少 | | 32×32 | 131,072 個 = 512 KB | 開始明顯 |

Tile 越大 → 搬運越少，但晶片內記憶體用越多。這就是 tile size sweep 實驗要量的 trade-off。

這是「解法」的原因

Tiling 不改變計算結果，只改變計算順序，卻讓搬運總量直接除以 4~16 倍。這是所有矩陣運算加速器（GPU 的 cuBLAS、TPU、LightNobel 的運算單元）都會用的基本手法。

2.5 解法二：寬位元與 burst → 提高每次搬運的效率 (v1 → v2)

白話直覺

v1 雖然減少了趟數，但推車很小，一次只能載 1 個 float (32 bit)。v2 換成大推車，一次載 16 個 float (512 bit)，並且 16 個廚師同時切。

兩個不同的概念：頻寬 vs 延遲

概念	意義	比喻
頻寬 (bandwidth)	每秒能搬多少 byte	高速公路有幾線道
延遲 (latency)	從發出請求到拿到第一筆資料要多久	從家裡開到倉庫要多久

一條 AXI 連接埠 (FPGA kernel 連到 HBM 的介面)：

頻寬上限 = 位元寬度 × 時脈
32-bit × 300 MHz = 1.2 GB/s (v1)
512-bit × 300 MHz = 19.2 GB/s (v2) ← 16 倍

Burst (連續搬運)

每次向 HBM 發出請求都要付一次延遲。如果要的資料在記憶體中是連續的，可以一次請求、連續拿很多筆，這叫 burst。

非 burst：請求 → 等 → 拿 1 筆 → 請求 → 等 → 拿 1 筆 → ... (每筆都付一次延遲)
Burst：請求 → 等 → 拿 1 筆、1 筆、1 筆、1 筆 ... (只付一次延遲)

這就是為什麼資料排列 (memory layout) 很重要：我們把資料排成 [row][col][channel]，同一個 token 的 128 個 channel 是連續的，所以一次可以 burst 讀完。v0 在最內層迴圈改變 k，每次跳過 128 個數字 (stride = C)，無法形成有效的 burst，這也是它慢的原因之一。

Little's Law：要多少資料「在路上」才能用滿頻寬

在途資料量 = 頻寬 × 延遲

例：頻寬 19.2 GB/s、延遲約 64 cycles (300 MHz 下約 213 ns) → 需要約 4 KB 的資料同時在路上，才能讓頻寬不空轉。一次只要一小筆、等到了才要下一筆，永遠用不滿頻寬。這也說明了 burst 和下一節 double buffering 的重要性。

這是「解法」的原因

寬位元讓每個 cycle 能搬的量變 16 倍；burst 讓延遲只付一次。兩者結合，同樣的搬運量所需的時間大幅下降。同時，因為一次拿到 16 個數，也能讓 16 個運算單元同時工作，運算能力也跟著提高 16 倍（v2 理論上限 = $16 \times 2 \times 300 \text{ MHz} = 9.6 \text{ GFLOP/s}$ ）。

2.6 解法三：Double buffering → 把等待時間藏起來 (v2 → v3)

白話直覺

v2 的廚師：搬一批 → 煮一批 → 搬下一批 → 煮下一批，搬的時候廚師在等。v3 多準備一個料理台：廚師煮第 k 批的同時，助手把第 k+1 批搬到另一個料理台。

v2（沒有重疊）：

時間 $\xrightarrow{\hspace{1.5cm}}$

[搬 k=0][算 k=0][搬 k=1][算 k=1][搬 k=2][算 k=2]

v3（double buffering, 兩組 buffer 輪流用）：

[搬 k=0][算 k=0][算 k=1][算 k=2][算 k=3]

 [搬 k=1][搬 k=2][搬 k=3]

 ↑ 搬運藏在計算底下

用實際數字看：v2 的 HLS 報告

每一輪 loop_k：| 步驟 | cycles | |---|---| | 載入 a、b（兩個並行） | 141 | | 計算（mac） | 526 | | 合計 | 約 671 |

套用 2.1 節的公式：

v2（輪流）： $T \approx T_{\text{compute}} + T_{\text{memory}} = 526 + 141 \approx 671 \text{ cycles}$

v3（重疊）： $T \approx \max(T_{\text{compute}}, T_{\text{memory}}) = \max(526, 141) \approx 530 \text{ cycles}$

時間減少 $\approx (671 - 530) \div 671 \approx 21\%$ → 速度約 1.27 倍

為什麼上限只有約 21%？

因為搬運只佔 21% 的時間。即使把搬運完全藏起來，最多也只能省下這 21%。這是 Amdahl's Law 的例子（見 2.10 節）。

延伸觀察：141 cycles 裡大部分是「延遲」

v2 每輪只搬 $2 \times 64 = 128$ 個 512-bit word（a、b 各 64 個，兩個 port 並行），理想上只要約 64 cycles，但報告顯示 141 cycles。多出的時間大多是 HBM 的延遲（HLS 預設假設 64 cycles）和迴圈的啟動成本。Double buffering 對付的正是這種「延遲型」的等待：延遲再長，只要能和計算重疊，就不會影響總時間。

這是「解法」的原因

Double buffering 不減少搬運量，也不提高頻寬，而是改變時間的結構：從「相加」變成「取最大值」。當計算和搬運的時間接近時，效果最好（最多快 2 倍）。

2.7 解法四：多通道 HBM → 多條路同時搬（cfg 對照實驗）

U55C 的 HBM 結構

16 GB HBM2 = 32 個 pseudo-channel (PC)，每個 512 MB、約 14 GB/s

PC0	PC1	PC2	...	PC31
-----	-----	-----	-----	------

總頻寬約 460 GB/s（全部同時用的情況下）

單一 PC 的頻寬和容量都不大，要同時使用多個 PC，才能發揮 HBM 的優勢。

本專題的兩種設定

設定檔	a	b	z	預期效果
trimul_v2.cfg	PC 0~7	PC 8~15	PC 16~23	三份資料走不同的路，互不干擾
trimul_v2_onebank.cfg	PC 0	PC 0	PC 0	三份資料搶同一條路

這是「解法」的原因

頻寬是「每條路的頻寬 × 同時使用的路數」。讓不同資料走不同的 PC，等於把公式中的 頻寬 乘上倍數；反之，全部擠在同一個 PC 會互相排隊。這個實驗需要上板才看得出效果（HLS 報告無法模擬 HBM 的真實行為）。

2.8 解法五：量化 → 讓每筆資料變小（v2 → v4）

白話直覺

把食材壓縮成原本的 1/4 再搬，搬到後再還原。會有一點失真，但搬運量少了 4 倍。

INT8 對稱量化的原理

把一組浮點數 x 轉成 -127~127 的整數 q ：

scale $s = \max(|x|) \div 127$ ← 讓最大的數剛好對應到 127
 $q = \text{round}(x \div s)$ ← 四捨五入成整數，存 1 byte
還原 $\hat{x} = s \times q$
誤差 $x - \hat{x}$ 落在 $[-s/2, +s/2]$ 之間

誤差的大小取決於 s : s 越大，每一格越粗，誤差越大。（在誤差均勻分布的假設下，均方根誤差約為 $s \div \sqrt{12} \approx 0.29 \times s$ 。）

為什麼 outlier 是問題：per-tensor vs per-token

我們的模擬資料：一般的數字大約落在 ± 3 以內，但有 1% 的 token 被放大 30 倍（outlier，大約 ± 80 ）。

Per-tensor（整份資料共用一把尺）：

$s = 84 \div 127 \approx 0.66$
 → 一般 token（大約 ± 3 ）只能用到約 $-4 \sim 4$ 這幾個整數
 → 均方根誤差 $\approx 0.29 \times 0.66 \approx 0.19$ ← 相對於數值本身（約 1）將近 20%！

Per-token（每個 token 各自一把尺）：

一般 token : $s \approx 2.8 \div 127 \approx 0.022 \rightarrow$ 均方根誤差 ≈ 0.0064 ($< 1\%$)
 outlier token : $s \approx 0.66$ ，但它自己的數值本來就大，相對誤差一樣很小

結論：per-tensor 時，少數 outlier 拖累了其他所有 token；per-token 讓每個 token 用適合自己的精度。我們的 C 模擬實測：per-token 輸出誤差約 1%，per-tensor 約 11%，與上面的推算方向一致。這正是 LightNobel「token-wise」量化的核心動機。

代價

項目	代價
精度	約 1% 的相對誤差 (per-token)
額外資料	每個 token 多存一個 scale (4 bytes / 128 bytes $\approx 3\%$)
額外運算	每次乘加要多乘上 $s_a \times s_b$ (v4 的 mac 迴圈)
量化本身	要先算出每個 token 的最大值 (我們在 host 端做，沒有計入 kernel 時間)

算術強度 (v4, 8×8 tile)

資料量 = $L^3 \cdot C \cdot (1/TI + 1/TJ) \times 1 \text{ byte}$ (int8)
 + $L^3 \cdot (1/TI + 1/TJ) \times 4 \text{ bytes}$ (scale)
 $AI = 2 \cdot L^3 \cdot C \div [L^3 \cdot (1/TI + 1/TJ) \cdot (C + 4)]$
 = $2 \cdot 128 \div [(1/4) \cdot 132] \approx 7.8 \text{ FLOP/byte}$ ← 約是 v2 的 3.9 倍

容量上的好處

版本	a, b 各佔	z	總計
FP32 (v2)	$L^2 \times 512 \text{ B}$	$L^2 \times 512 \text{ B}$	$L^2 \times 1,536 \text{ B}$
INT8 (v4)	$L^2 \times 132 \text{ B}$ (含 scale)	$L^2 \times 512 \text{ B}$ (輸出仍是 FP32)	$L^2 \times 776 \text{ B}$

總佔用量約減半 → 相同記憶體下， L 約可增加 $\sqrt{2} \approx 1.4$ 倍。（若輸出也量化，才會接近 2 倍，可以當延伸題目。）

這是「解法」的原因

量化同時縮小兩件事：搬運量（公式中的分子）和記憶體佔用（決定 L 的上限）。這是唯一一個直接處理「序列長度受限」的手法，也是 LightNobel 的核心。

2.9 其他常見手法（本專題未實作，但報告中可以提）

手法	原理	代價	在 PPM 的例子
Chunking (分塊)	一次只算一部分，算完就丟	時間變長	AlphaFold2/ESMFold 內建 chunk size 設定
Recomputation (重算)	不存中間值，需要時重新算	多花計算	訓練時常用的 activation checkpointing
Operator fusion (運算融合)	把連續幾個運算合成一個，中間值不寫回記憶體	程式較複雜	LayerNorm + Linear + gating 合併
低精度浮點 (FP16/BF16)	每個數用 2 bytes	範圍或精度下降	語言模型部分常用 FP16
稀疏化	跳過接近 0 的值	需要不規則存取	距離很遠的 (i, j) 對

2.10 為什麼「效益可能不大」仍然算解法？

Amdahl's Law

$$\text{整體加速} = 1 \div [(1 - p) + p \div s]$$

p = 被改善的部分佔原本時間的比例

s = 那部分被加速的倍數

例：v3 只改善「等記憶體」那 21% ($p = 0.21$)，即使讓它完全消失 ($s \rightarrow \infty$)：

$$\text{整體加速} = 1 \div (1 - 0.21) \approx 1.27 \text{ 倍}$$

瓶頸轉移

每解決一個瓶頸，下一個瓶頸就會浮現：

v0：受限於記憶體 ($AI = 0.25$)

→ v1 tiling：搬運量減少 8 倍，但每 cycle 只做 1 次乘加 → 變成受限於運算

→ v2 寬位元 + 16 路平行：運算變 16 倍 → 記憶體延遲佔 21%

→ v3 double buffering：延遲被藏起來 → 又變成受限於運算 (16 路)

→ 下一步：增加平行度（例如展開 jj 迴圈）→ 又會回到受限於記憶體 ...

找出每一步的瓶頸在哪、為什麼轉移，本身就是有價值的分析。這也是為什麼報告中要畫 roofline：可以看到 kernel 在圖上一步步移動。

什麼叫「有解決到問題」

問題	手法	從什麼概念上解決
搬運太多	Tiling	利用資料重用，減少重複搬運
搬運沒效率	寬位元、burst	每次搬更多、延遲只付一次
等待延遲	Double buffering	讓搬運和計算重疊
頻寬不夠	多通道	同時使用多條路
資料太大、放不下	量化	每筆資料變小，同時降低容量與頻寬需求

本章小結

- 記住這條公式： $T \approx T_{\text{compute}} + T_{\text{memory}}$ （沒重疊）或 $\max(\dots)$ （有重疊），每一招都在縮小其中一項。
- 用算術強度判斷 memory-bound 還是 compute-bound；用 roofline 圖呈現結果。
- 本專題五個版本的 AI：0.25 → 2 → 2 → 2 → 約 7.8 (FLOP/byte)。
- Double buffering 的上限由搬運佔比決定（v2 約 21%），這是 Amdahl's Law。
- 量化是唯一同時處理「頻寬」和「容量（序列長度）」的手法，per-token 可以避免 outlier 拖累精度。

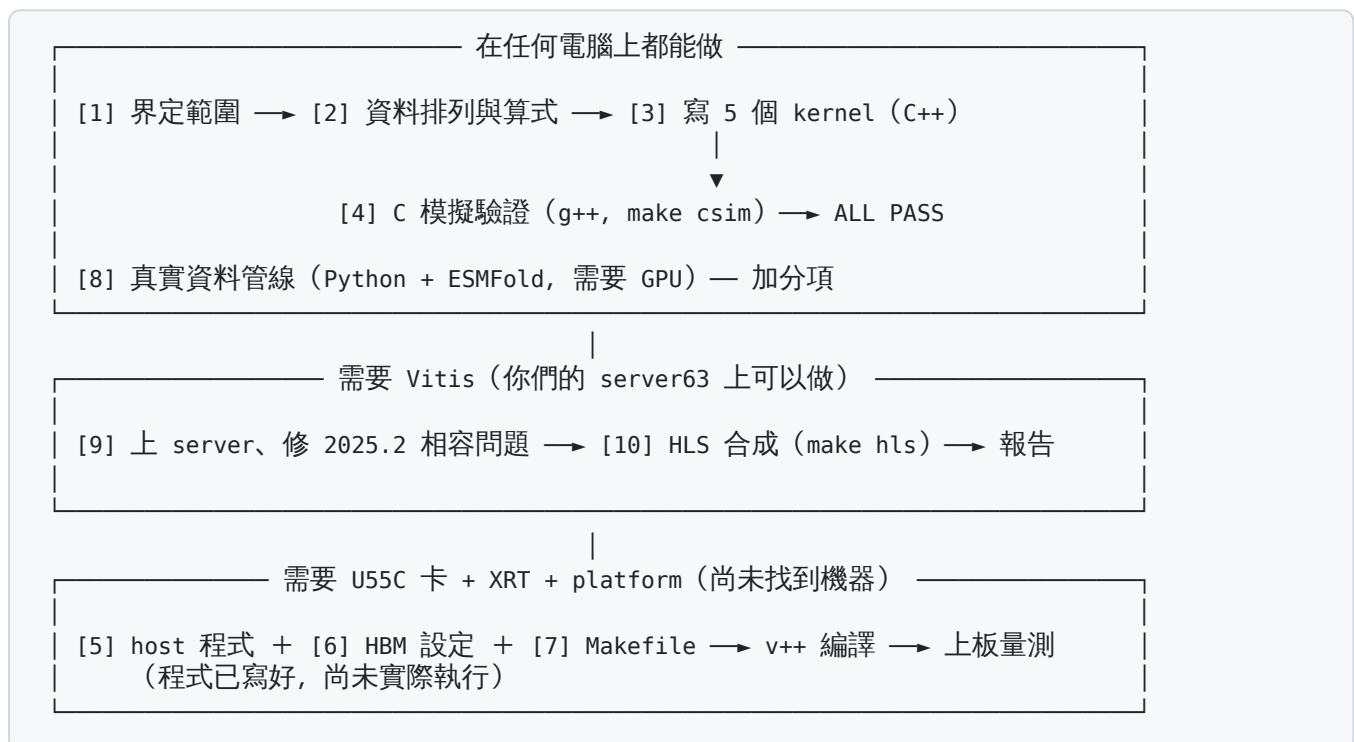
下一章會一步一步回顧我們實際做了什麼、每一步為什麼做、產生了哪些檔案。

第 3 章 整體流程：從頭到現在具體做了什麼？

本章重點

- 我們的工作分成十個步驟：界定範圍 → 資料排列 → 寫 kernel → 驗證 → host → HBM 設定 → Makefile → 真實資料 → 上 server 除錯 → 讀報告。
- 每一步都說明：為什麼做、怎麼做、有什麼效果、產生了什麼檔案。
- 最後有一張所有檔案的總表。

3.1 全景圖



3.2 步驟一：界定範圍

為什麼做

LightNobel 處理的是整個 PPM，還設計了專用硬體。專題要求「用常見手法比較」，所以要從中挑出一個能代表問題、又做得完的小範圍。

怎麼決定

考量	Triangle Multiplication 的情況
能代表問題嗎？	✅ 直接處理 $L \times L \times 128$ 的 pair representation

考量	Triangle Multiplication 的情況
算式簡單、容易驗證嗎？	✓ 本質是 128 個矩陣乘法
能展示多種記憶體手法嗎？	✓ tiling、寬位元、重疊、多通道、量化都適用
能呼應 LightNobel 嗎？	✓ 可以比較 per-token vs per-tensor 量化

產出

一句話的題目定義（見第 1 章 1.7 節），以及「只做 einsum 核心，不含 LayerNorm/gating」的範圍界定。

3.3 步驟二：資料排列（memory layout）與算式

為什麼做

資料在記憶體中怎麼排，決定了能不能做 burst（連續讀取，見第 2 章 2.5 節）。

我們的選擇

a、b、z 都是 $L \times L \times C$ ，排成 [row][col][channel]：
 $\text{index} = (\text{row} \times L + \text{col}) \times C + c$

記憶體中的樣子（ $C = 128$ ）：

(0,0) 的 128 個數	(0,1) 的 128 個數	...	(0,L-1) 的 128 個數	(1,0) ...
----------------	----------------	-----	------------------	-----------

↑ 同一個 token 的 128 個 channel 是連續的 → 可以一次 burst 讀完

效果

- 每個 token 的 128 個 float = 512 bytes = 剛好 8 個 512-bit word，完美對齊寬位元傳輸。
- 這也是 PyTorch 中 ESMFold 的預設排列（最後一維是 channel），所以從 Python 擷取的資料可以直接使用，不用轉換。

產出

fpga/trimul/kernels/trimul.h：定義 $C = 128$ 、tile 大小、f16（16 個 float 的 struct）、q64（64 個 int8 的 struct）等共用設定。

3.4 步驟三：寫 5 個 kernel

以下擷取每個版本最關鍵的部分，完整程式碼在 fpga/trimul/kernels/。

v0 : 最直白的寫法 (`trimul_v0.cpp`)

```
for (int i = 0; i < L; i++)
  for (int j = 0; j < L; j++)
    for (int c = 0; c < C; c++) {
      float acc = 0.0f;
      for (int k = 0; k < L; k++) {
#pragma HLS PIPELINE
        acc += a[(i * L + k) * C + c] * b[(j * L + k) * C + c];    // 每次都讀 HBM
      }
      z[(i * L + j) * C + c] = acc;
    }
}
```

問題在哪：

1. 每次乘加都從 HBM 讀兩個數，完全沒有重用。
2. 最內層的 k 每次跳 C 個位置 (stride = 128)，無法 burst。
3. `acc += ...` 下次要等上一次加完 (浮點加法需要好幾個 cycle)，pipeline 無法每 cycle 開始一次。

v1 : Tiling (`trimul_v1.cpp`)

```
float acc[TI][TJ][C];          // 累加器留在晶片內 (BRAM)
float la[TI][C], lb[TJ][C];    // 本輪用到的 a、b

for (i0 ...; i0 += TI)          // 一次處理 TI 列
  for (j0 ...; j0 += TJ) {      // 和 TJ 行
    acc = 0
    for (int k = 0; k < L; k++) {
      load_a: la[ii][c] = a[(i0 + ii) * L + k] * C + c];    // 搬 TI×C 個 (連續)
      load_b: lb[jj][c] = b[(j0 + jj) * L + k] * C + c];    // 搬 TJ×C 個 (連續)
      mac:   acc[ii][jj][c] += la[ii][c] * lb[jj][c];        // TI×TJ×C 次乘加
    }
    store: z[...] = acc[ii][jj][c];
  }
}
```

改善：每個 `la` 用 TJ 次、每個 `lb` 用 TI 次 (搬運量減 8 倍)；每次載入 C 個連續的數 (可以 burst)。

一個細節：`mac` 迴圈裡加了

```
#pragma HLS DEPENDENCE variable = acc inter false
```

意思是告訴 HLS：「同一個 `acc` 位置要隔 $TI \times TJ \times C = 8192$ 次迭代才會再被更新，不用擔心前後相依」，這樣 HLS 才敢讓迴圈每個 cycle 開始一次新的迭代 ($II = 1$)。

v2 : 寬位元 + 16 路平行 (`trimul_v2.cpp`)

```
struct f16 { float v[16]; };    // 一個 512-bit word

float acc[TI * TJ * CW][LANES]; // LANES = 16, CW = C/16 = 8
#pragma HLS ARRAY_PARTITION variable = acc dim = 2 complete // 16 個 lane 拆開成獨立的記憶體體

mac:
for (int r = 0; r < TI * TJ * CW; r++) {    // 512 次
#pragma HLS PIPELINE II = 1
    for (int l = 0; l < LANES; l++)          // 這層會被完全展開 → 16 個乘加器同時做
        acc[r][l] += la[...][l] * lb[...][l];
}
```

改善：

- 介面從 `float*` 改成 `f16*` → 每次讀 512 bit (16 個 float)。
- `ARRAY_PARTITION` 把 16 個 lane 拆到不同的記憶體區塊，才能同一個 cycle 讀寫 16 個數。
- 內層 `for l` 在 PIPELINE 裡會被自動展開成 16 份硬體。

v3 : Double buffering (`trimul_v3.cpp`)

```
float la0[...], lb0[...]; // 第一組 buffer
float la1[...], lb1[...]; // 第二組 buffer

load_ab(k = 0 → la0, lb0);
for (int k = 0; k < L; k++) {
    if (k % 2 == 0) {
        if (k + 1 < L) load_ab(k + 1 → la1, lb1); // 搬下一批到 buffer 1
        mac(la0, lb0, acc);                        // 同時用 buffer 0 計算
    } else {
        if (k + 1 < L) load_ab(k + 1 → la0, lb0); // 搬下一批到 buffer 0
        mac(la1, lb1, acc);                        // 同時用 buffer 1 計算
    }
}
```

關鍵：`load_ab` 和 `mac` 是兩個獨立的函式 (`#pragma HLS INLINE off`)，而且使用不同的 buffer、彼此沒有資料相依，所以 HLS 可以讓它們同時執行。

v4 : INT8 per-token 量化 (`trimul_v4.cpp`)

```
struct q64 { int8_t v[64]; };    // 一個 512-bit word = 64 個 int8

// 每個 token 有自己的 scale : sa[i*L + k]、sb[j*L + k]
mac:
for (int r = 0; r < TI * TJ * QW; r++) {
#pragma HLS PIPELINE II = 1
    float s = lsa[ii] * lsb[jj];           // 兩個 token 的 scale 相乘
    for (int l = 0; l < 64; l++) {         // 64 路平行
        int16_t p = (int16_t)la[..][l] * (int16_t)lb[..][l]; // int8 × int8 → int16
        acc[r][l] += (float)p * s;        // 還原尺度後累加
    }
}
```

改善：每次搬 64 個數 (FP32 時只有 16 個)，搬運量變 1/4。為什麼 scale 不能提到迴圈外：每個 k 的 token 不同，scale 也不同，所以每次都要乘。（如果改成 per-channel 量化，scale 就能提到外面，用純整數累加，但就不是 token-wise 了。）

產出

`kernels/trimul_v0.cpp ~ trimul_v4.cpp`，以及共用的 `trimul.h`。

3.5 步驟四：正確性驗證 (C 模擬)

為什麼做

上板編譯一次要 1~數小時。先用幾秒鐘確認程式算得對，才不會浪費時間。

怎麼做 (`tb/tb.cpp`)

- 產生隨機資料 `a`、`b` (`host/common.h` 的 `make_activation`)：- 一般 token：常態分布 $N(0, 1)$ - 1% 的 token 放大 30 倍：模擬真實 activation 中的 outlier
- 依序呼叫 `v0~v4`，把結果和 CPU 用 double 精度算出的正確答案比較。
- 印出兩種誤差： $rel_l2 = \|z - z_ref\| \div \|z_ref\|$ （整體相對誤差） $max_abs = \max |z - z_ref|$ （最大的單點誤差）
- `v0~v3` 要求 $rel_l2 < 1e-5$ （只有浮點誤差）；`v4` 允許較大誤差（量化本來就有誤差）。

結果

v0 naive	rel_l2=1.141e-07	PASS	
v1 tiled	rel_l2=1.141e-07	PASS	
v2 tiled+wide	rel_l2=1.141e-07	PASS	
v3 tiled+wide+pingpong	rel_l2=1.141e-07	PASS	
v4 int8 per-token	rel_l2=9.871e-03	PASS	← 約 1%
v4 int8 per-tensor	rel_l2=1.126e-01	PASS	← 約 11%

- `v0~v3` 的誤差完全一樣 (1.1e-7)，代表改寫沒有改變計算結果。

- per-tensor 誤差約是 per-token 的 11 倍，這是本專題第一個可以放進報告的結論。

為什麼 g++ 能跑 HLS 程式？

因為 kernel 就是普通的 C++。#pragma HLS ... 對 g++ 來說是不認識的指示，會被忽略（我們加了 -Wno-unknown-pragmas 讓它不要警告）。詳見第 4 章 4.4 節。

產出

tb/tb.cpp、host/common.h；指令 make csim。

3.6 步驟五：host 程式

為什麼做

FPGA 不會自己去拿資料。需要一個在 CPU 上執行的程式負責：準備資料 → 送進 FPGA 的 HBM → 啟動 kernel → 等它算完 → 取回結果 → 檢查、計時。

流程（host/host.cpp，使用 XRT 的 C++ API）

```
xrt::device dev(0); // 1. 打開第 0 張 FPGA 卡
auto uuid = dev.load_xclbin("trimul_v2.xclbin"); // 2. 把編譯好的電路燒進去
xrt::kernel krnl(dev, uuid, "trimul_v2"); // 3. 找到 kernel

xrt::bo bo_a(dev, bytes, krnl.group_id(0)); // 4. 在 HBM 上配置記憶體
bo_a.write(a.data()); // (group_id 決定放在哪個 PC)
bo_a.sync(XCL_BO_SYNC_BO_TO_DEVICE); // 5. 從 CPU 記憶體複製到 HBM

run.start(); run.wait(); // 6. 啟動 kernel 並等待完成（計時）

bo_z.sync(XCL_BO_SYNC_BO_FROM_DEVICE); // 7. 把結果複製回 CPU
bo_z.read(z.data());
```

最後印出時間、GFLOP/s、估計的搬運量和頻寬、HBM 佔用量、誤差，以及一行 CSV 方便整理。

狀態

⚠ 已寫好，並用假的 XRT header 確認可以編譯，但尚未在真的板子上執行過。

產出

host/host.cpp；指令 make host、make run。

3.7 步驟六：HBM 連接設定

為什麼做

kernel 有好幾個 AXI 連接埠 (a、b、z……)，每個要接到 HBM 的哪個 pseudo-channel，是在連結 (link) 階段由設定檔決定的。

設定檔 (`cfg/trimul_v2.cfg`)

```
[connectivity]
sp=trimul_v2_1.a:HBM[0:7]      # a 接到 PC 0~7 (共 4 GB)
sp=trimul_v2_1.b:HBM[8:15]    # b 接到 PC 8~15
sp=trimul_v2_1.z:HBM[16:23]   # z 接到 PC 16~23
```

- `trimul_v2_1`：kernel 的第一個實例 (instance)。
- `sp`：system port，指定「kernel 的某個參數要接到哪塊記憶體」。
- 對照組 `trimul_v2_onebank.cfg`：全部接到 `HBM[0]`。

產出

`cfg/trimul_v0.cfg` ~ `trimul_v4.cfg`、`trimul_v2_onebank.cfg`。

3.8 步驟七：Makefile

為什麼做

完整流程有很多步驟、每個指令都很長。Makefile 把它們包成短指令，參數也集中管理。

指令一覽

指令	做什麼	需要什麼
<code>make csim</code>	g++ 編譯並執行 testbench	只要 g++
<code>make hls KERNEL=trimul_v2</code>	執行 <code>hls/run_hls.tcl</code> ，產生合成報告	Vitis
<code>make hls-all</code>	依序合成 v0~v4，最後呼叫 <code>make summary</code>	Vitis
<code>make summary</code>	執行 <code>scripts/hls_summary.py</code> ，產生比較表與 <code>hls_summary.csv</code>	Python
<code>make xclbin KERNEL=... TARGET=hw</code>	<code>v++ -c</code> (編譯) + <code>v++ -l</code> (連結) → <code>.xclbin</code>	Vitis + U55C platform
<code>make host</code>	編譯 host 程式	XRT
<code>make run ...</code>	執行 (emulation 或上板)	XRT (上板還需要卡)

可調參數

`KERNEL`、`TARGET` (`sw_emu` / `hw_emu` / `hw`)、`PLATFORM`、`CFG`、`DEFS` (例如 `tile` 大小)、`TAG`、`RUN_L`、`DATA`。

產出

`Makefile`、`hls/run_hls.tcl`、`scripts/hls_summary.py`、`.gitignore`。

3.9 步驟八：真實資料管線（加分項）

為什麼做

模擬資料的 outlier 是我們「假設」的。要證明結論在真實蛋白質上也成立，需要從 ESMFold 取出真正的 activation。

怎麼做（`python/dump_trimul_inputs.py`）

1. 下載序列（UniProt / PDB）或讀 FASTA。
2. 載入 HuggingFace 的 ESMFold（`facebook/esmfold_v1`）。
3. 把 `trunk.blocks[N].tri_mul_out.combine_projections` 換成我們的函式：當模型執行到這裡時，把 `a`、`b` 存下來，然後丟出例外讓模型停止（後面的 block 不用跑，省時間）。
4. 計算 outlier 程度、per-token / per-tensor 的量化誤差。
5. 補零到 32 的倍數，存成 `a.bin`、`b.bin`（float32，排列和 kernel 相同）。

驗證：「差值 0」是怎麼來的

在隨機初始化的小型 ESMFold 上，擷取 `a`、`b` 後比較：

```
ref = tm._combine_projections(a, b)      # ESMFold 自己的計算
ours = torch.einsum("ikc,jkc->ijc", a, b) # 我們 kernel 的算式
max diff = 0.0
```

證明我們的 kernel 算的確實是 ESMFold 裡的那個運算。

過程中發現的錯誤

一開始用 tokenizer 產生輸入，結果模型報錯（index out of range）。原因是 ESMFold 的 `forward` 需要的是 AlphaFold 的胺基酸編號，不是語言模型 tokenizer 的編號；要用 `model.infer(seq)` 才會正確轉換。

狀態

需要 GPU（或 Colab）才能跑正式模型；使用者沒有 GPU，所以目前是加分項。C 模擬、host 程式和 Makefile 都已支援讀取這些檔案（`DATA=...`）。

產出

python/dump_trimul_inputs.py、python/requirements.txt、docs/real_protein_data.md。

3.10 步驟九：在實際 server 上除錯

環境調查

指令	結果	結論
<code>which vivado vitis v++</code>	都在 <code>/tools/Xilinx/2025.2/</code>	工具齊全，版本很新
<code>which vitis_hls</code>	找不到	2025.2 已移除，改用 <code>vitis-run</code>
<code>lspci -d 10ee:</code>	空的	這台沒有 FPGA 卡（10ee 是 Xilinx 的廠商代號）
<code>xbutil</code> 、 <code>/opt/xilinx/platforms</code>	沒有	沒有 XRT、沒有 U55C platform

→ 這台可以做 C 模擬和 HLS 合成，不能上板。

除錯過程：一次失敗、三次修正

這是一個很好的除錯範例，展示「從錯誤訊息推理出原因」的過程。

第 1 次失敗

```
ld.lld: error: undefined symbol: trimul_v2
```

- 觀察：編譯清單裡有 `tb`、`v0`、`v1`、`v3`、`v4`，唯獨沒有 `v2`。
- 當時的判斷：2025.2 的 C 模擬流程有問題 → 改成預設跳過 HLS 內建的 C 模擬（反正 `make csim` 已經驗證過）。
- （後來發現真正原因和第 2、3 次相同。）

第 2 次失敗

```
WARNING: Cannot find source file ../kernels/trimul_v2.cpp; skipping it.  
ERROR: Cannot find any design unit to elaborate.
```

- 觀察：工具找不到來源檔。
- 假設：相對路徑的問題 → 改用絕對路徑。

第 3 次失敗（相同錯誤）

- 關鍵觀察：日誌中 Adding design file `'/home/anthony/.../trimul_v2.cpp'` 已經是絕對路徑，但到了合成階段，工具又自己改寫成 `../kernels/trimul_v2.cpp`。
- 推理：專案位置：`fpga/trimul/hls_prj/trimul_v2/`（在目前資料夾往下 2 層）正確的相對路徑：`../../kernels/trimul_v2.cpp` 工具算出來的：`../kernels/trimul_v2.cpp`（只往上 1 層）

→ 工具假設專案只在目前資料夾的下一層。專案名稱含有 / (hls_prj/trimul_v2) 時就會算錯。

- 修正：先 `cd hls_prj`，再建立名為 `trimul_v2` 的專案（只差一層）。

第 4 次：成功 ✓

學到的除錯方法

1. 仔細看完整的錯誤訊息，不只看 ERROR，也看前面的 WARNING。
2. 比較「預期」和「實際」（編譯清單少了一個檔案、路徑少了一層）。
3. 修正後結果不變 → 假設錯了，回頭找新的線索。
4. 用算的驗證假設（數路徑的層數）。

產出

修改過的 `Makefile`（自動選 `vitis_hls` 或 `vitis-run`）與 `hls/run_hls.tcl`。

3.11 步驟十：讀懂 HLS 報告（以 v2 為例）

報告位置：`hls_prj/trimul_v2/sol/syn/report/trimul_v2_csynth.rpt`

① 基本資訊

```
* Version:      2025.2
* Solution:      sol (Vitis Kernel Flow Target)
* Product family: virtexuplusHBM
* Target device: xcu55c-fsvh2892-2L-e      ← U55C 的晶片型號
```

② Timing（時脈）

```
| Clock | Target | Estimated | Uncertainty |
| ap_clk | 3.33 ns | 2.431 ns | 0.90 ns |
```

欄位	意義
Target	我們要求的時脈週期 (3.33 ns = 300 MHz)
Estimated	HLS 估計電路最慢的路徑需要多久
Uncertainty	保留給後續繞線的餘裕

判斷：Estimated + Uncertainty = 3.33 ns，剛好在目標內 → 通過。（最終能否達到 300 MHz，要等 Vivado 實際繞線後才知道。）

③ Latency（總時間）

```
| Latency (cycles) | Latency (absolute) | Interval |
| 177,029,185      | 0.590 sec          | 177029186 |
```

- Latency：從開始到結束的 cycle 數。
- absolute：cycle 數 × 時脈週期。
- Interval：可以接受下一次呼叫的間隔（這裡和 latency 差不多，因為一次只跑一個）。
- ⚠ 這是在 $L = 256$ 下的估計（由 trimul.h 的 L_TRIP 決定）。因為 L 是執行時才知道的參數，HLS 需要 LOOP_TRIPCOUNT 告訴它迴圈大概跑幾次，才能估算。

④ Instance（子模組）

Module	Latency (cycles)	
trimul_v2_Pipeline_init	514	← 清空累加器（512 次 + 啟動）
trimul_v2_Pipeline_load_a	141	← 載入 a
trimul_v2_Pipeline_load_b	141	← 載入 b
trimul_v2_Pipeline_store	586	← 寫回結果
trimul_v2_Pipeline_mac	526	← 計算（512 次 + pipeline 深度 14）

每個 pipeline 化的迴圈會變成一個獨立的硬體模組。

⑤ Loop（迴圈）

Loop Name	Latency	Iteration Latency	Trip Count
tile_i	177,029,184	5,532,162	32
tile_j	5,532,160	172,880	32
loop_k	171,776	671	256

驗算：

```

loop_k : 256 次 × 671 = 171,776 ✓
tile_j 每次 : 171,776 (loop_k) + 514 (init) + 586 (store) + 4 ≈ 172,880 ✓
tile_i 每次 : 32 × 172,880 + 2 = 5,532,162 ✓
總計 : 32 × 5,532,162 ≈ 177,029,184 ✓

```

loop_k 的 671 cycles 就是第 2 章 2.6 節分析 double buffering 的依據：load（141, a、b 並行）+ mac（526）+ 少量 ≈ 671。

⑥ 換算效能

```

運算量 =  $2 \times 256^3 \times 128 \approx 4.29 \times 10^9$  FLOP
時間    = 0.590 s
效能    ≈ 7.3 GFLOP/s          (v2 理論上限 9.6 GFLOP/s 的 76%)

```

⑦ 資源用量（報告的下一段，尚未截圖）

Utilization Estimates 段落會列出 BRAM_18K、DSP、FF、LUT、URAM 的用量和百分比。make summary 會自動把這些數字整理進比較表。

3.12 所有檔案總表

檔案	用途	狀態
README.md	repo 目錄	✓
docs/PROJECT_LOG.md	專題工作紀錄 (交接用)	✓
docs/conversation_notes.md	白話筆記	✓
docs/getting_started_u55c.md	U55C 環境與開發流程	✓
docs/real_protein_data.md	真實資料流程	✓
docs/manual/	本教學手冊 (含 manual.pdf)	✓
fpga/trimul/kernels/trimul.h	共用設定	✓
fpga/trimul/kernels/trimul_v0~v4.cpp	5 個 kernel	✓ csim ; v2 已合成
fpga/trimul/tb/tb.cpp	C 模擬 testbench	✓
fpga/trimul/host/common.h	資料產生、量化、參考答案、讀檔	✓
fpga/trimul/host/host.cpp	XRT host 程式	⚠ 未上板
fpga/trimul/cfg/*.cfg	HBM 連接	⚠ 未使用
fpga/trimul/hls/run_hls.tcl	HLS 腳本	✓ (2025.2)
fpga/trimul/scripts/hls_summary.py	報告整理成表	✓ (用假報告測過)
fpga/trimul/Makefile	指令捷徑	✓ csim/hls ; ⚠ xclbin/run
python/dump_trimul_inputs.py	擷取真實 activation	✓ (小模型測過)

執行後會產生 (不進 git) : hls_prj/ (HLS 專案與報告)、hls_*.log、hls_summary.csv、build/ (v++ 產物)、*.exe。

本章小結

- 流程的核心精神：從快到慢驗證 (C 模擬 → HLS → emulation → 上板)，前一步沒過就不進下一步。
- 每個 kernel 版本只改一件事，才能清楚歸因每一招的效果。
- 除錯時，比較預期與實際、修正無效就換假設。
- 目前進度：C 模擬全部通過、v2 合成成功，正在跑 v0~v4 全部合成；上板部分尚待找到有 U55C 的機器。

下一章解釋這些工具背後的原理：為什麼 C++ 可以變成電路、每個工具實際做了什麼、這些能力從哪裡來。

第 4 章 工具原理：為何 C++ 可以變成電路？

本章重點

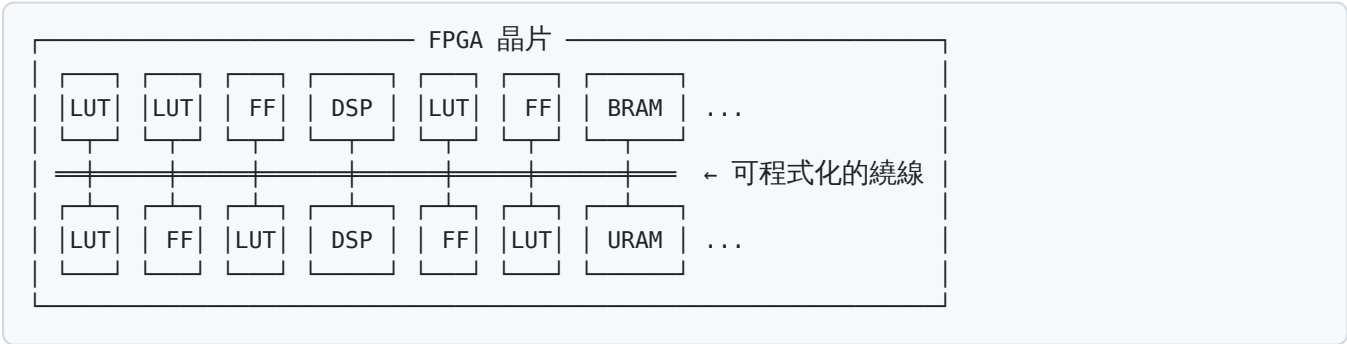
- FPGA 是一塊「可以重新接線的晶片」：裡面有大量可設定的小元件（LUT、FF、DSP、BRAM），靠一份設定檔（bitstream）決定它們怎麼連接。
- 把 C++ 變成電路的是 AMD 的 Vitis HLS，不是 Claude。它把程式拆成運算、排進時脈週期、對應到硬體元件，最後輸出 Verilog。
- C++ 可以先模擬，是因為 C++ 程式本身就是功能規格；HLS 保證產生的電路和程式算出一樣的結果。
- 整條工具鏈：g++（驗證）→ Vitis HLS（C++ → RTL）→ Vivado（RTL → bitstream）→ v++（接上 platform）→ XRT（host 控制板子）。
- 最後一節清楚劃分：哪些是下載／安裝的工具、哪些是 Claude 寫的、Claude 在這個對話中實際執行了什麼。

4.1 FPGA 是什麼？

白話直覺

- CPU 像一位「什麼都會做的通才」：硬體固定，靠執行不同的指令完成不同工作。
- GPU 像「一大群做同樣動作的工人」：硬體固定，擅長大量重複的計算。
- ASIC（例如 LightNobel 設計的加速器）像「為一道菜專門打造的生產線」：最快、最省電，但做好就不能改，而且製造非常貴。
- FPGA 像「一大箱樂高」：可以依需求組成任何生產線，組錯了拆掉重來。速度介於 CPU/GPU 和 ASIC 之間。

FPGA 裡面有什麼



元件	是什麼	白話	U55C 的數量
LUT (Look-Up Table)	一個 6 輸入的小真值表，可以實現任何 6 個輸入的邏輯函數	可以變成任何小邏輯閘組合	約 130 萬

元件	是什麼	白話	U55C 的數量
FF (Flip-Flop)	1 bit 的記憶元件，每個時脈更新一次	暫存器，用來「記住上一步的結果」	約 260 萬
DSP (DSP48E2)	一個 27×18 bit 的硬體乘法器 + 累加器	專門做乘法的小計算機	9,024
BRAM	36 Kb 的小記憶體（可拆成兩塊 18 Kb），有 2 個讀寫埠	料理台	2,016 (36 Kb)
URAM	288 Kb 的較大記憶體	大料理台	960
繞線 (routing)	可程式化的連線網路	決定誰接到誰	—

LUT 為什麼能變成「任何邏輯」

LUT 本質上是一塊 64 格的小記憶體 ($2^6 = 64$)。6 個輸入當作「地址」，存在裡面的 0/1 就是輸出。

例：想做「3 個輸入的多數決」（兩個以上是 1 就輸出 1）

輸入 abc：000 001 010 011 100 101 110 111

輸出：0 0 0 1 0 1 1 1 ← 把這 8 個 bit 寫進 LUT 就完成了

改寫 LUT 裡的內容，就改變了電路的功能。這就是「可程式化」的本質。

Bitstream

Bitstream 是一個檔案，裡面記錄了：每個 LUT 要存什麼、每條繞線要不要接通、每個 DSP 和 BRAM 怎麼設定。把 bitstream 寫進 FPGA，它就變成我們設計的電路。在 Vitis 流程中，bitstream 被包在 **.xclbin** 檔案裡。

一個浮點乘法在 FPGA 上長什麼樣子

float x, y → [拆出正負號、指數、尾數 (LUT)]



[尾數相乘 (約 2~3 個 DSP)]



[指數相加、正規化、捨入 (LUT + FF)]



x × y

為了跑到 300 MHz，中間會插入好幾級 FF，所以一個浮點乘法需要好幾個 cycle 才出結果。

v2 的 16 路平行，就是真的放了 16 組這樣的硬體。

4.2 為什麼 FPGA 適合做記憶體優化的實驗？

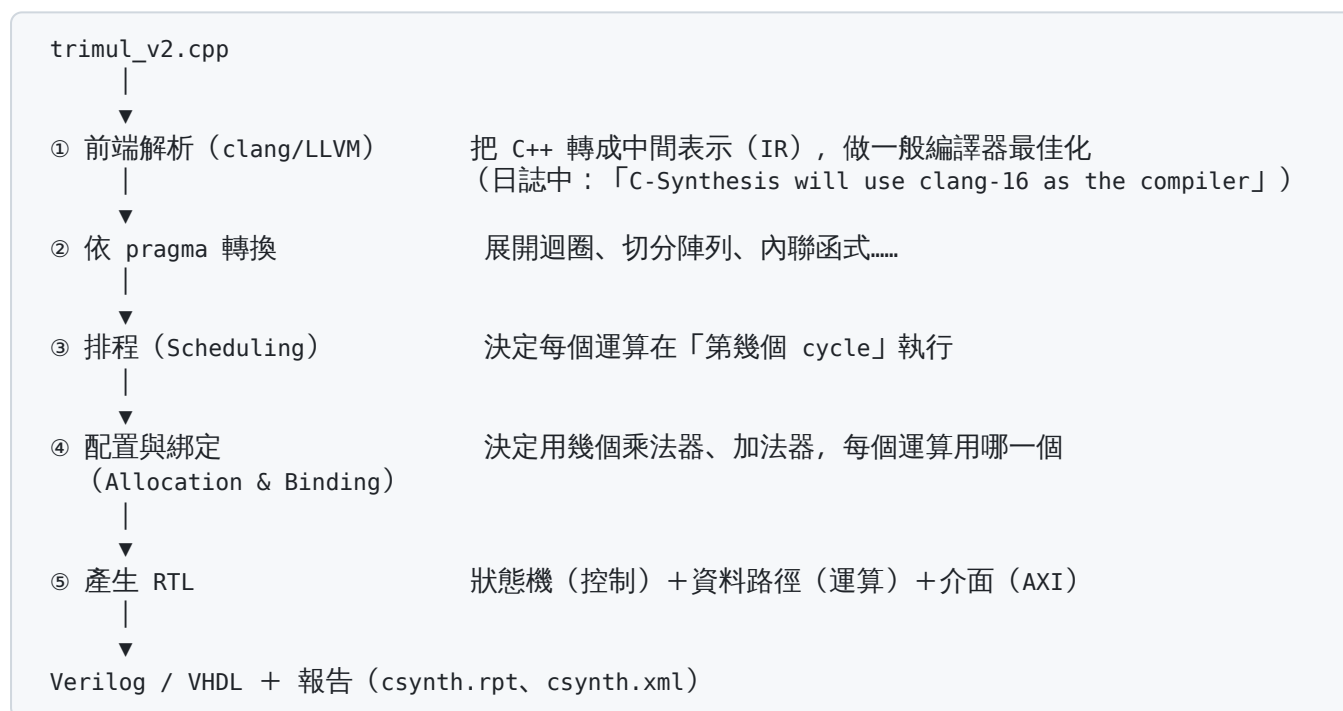
特性	CPU / GPU	FPGA
記憶體階層	固定 (cache 由硬體自動管理)	自己設計 (要多少 buffer、放在哪、怎麼切都可以決定)
資料怎麼搬	由硬體和驅動程式決定	自己決定 (位元寬度、burst 長度、哪個通道)
運算單元數量	固定	自己決定 (16 路、64 路……)
能看到多少細節	很多被隱藏	報告直接列出每個迴圈、每種資源

這讓我們能「一次只改一件事」，清楚量出每一招的效果。

4.3 HLS : C++ 是怎麼變成電路的？

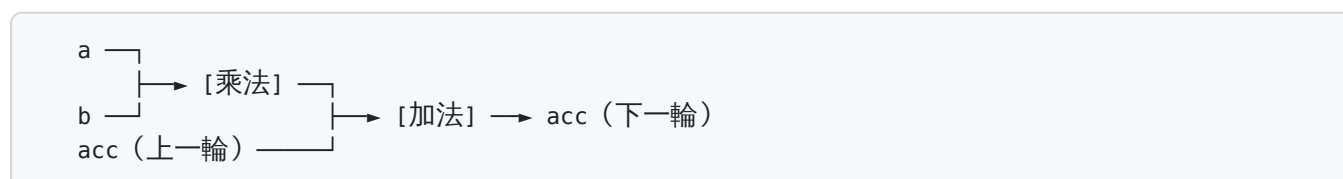
HLS (High-Level Synthesis, 高階合成) : 把 C/C++ 自動翻譯成硬體描述語言 (Verilog/VHDL)。本專題用的是 AMD 的 Vitis HLS (2025.2 以 `vitis-run --mode hls` 執行)。

整體流程



① 前端解析：把程式拆成「運算的圖」

例：`acc += a * b;`



這張圖叫 資料流圖 (data flow graph)：節點是運算，箭頭是資料相依。

③ 排程：把運算放進時脈週期

每種運算在 300 MHz 下需要的 cycle 數不同 (例如浮點乘法、浮點加法都要好幾個 cycle)。HLS 依照資料相依關係，把每個運算排到某個 cycle：

cycle:	1	2	3	4	5	6	7	8	9	10
讀 a、b	█									
乘法		█	█	█	█					
加法					█	█	█	█		
寫 acc									█	

一次迭代從開始到結束需要的 cycle 數叫 iteration latency。

Pipeline 與 II (Initiation Interval)

沒有 pipeline：一次迭代做完才開始下一次。

迭代 0：	[讀][乘乘乘][加加加加][寫]
迭代 1：	[讀][乘乘乘][加加加加][寫]

有 pipeline (II = 1)：每個 cycle 都開始一個新的迭代，像工廠的流水線。

迭代 0：	[讀][乘乘乘][加加加加][寫]
迭代 1：	[讀][乘乘乘][加加加加][寫]
迭代 2：	[讀][乘乘乘][加加加加][寫]
迭代 3：	[讀][乘乘乘][加加加加][寫]

II = 連續兩次迭代開始的間隔。II = 1 代表每個 cycle 都有一個結果產出，是最理想的狀態。

例：v2 的 mac 迴圈 跑 512 次，報告顯示 526 cycles：

$526 \approx 512$ (每 cycle 開始一次, II = 1) + 14 (pipeline 的深度, 最後一次迭代要走完)

為什麼 v0 無法達到 II = 1

v0 的 `acc += a * b`：下一次的加法需要這一次加完的結果。

迭代 0：	[加加加加]
迭代 1：	[加加加加] ← 要等迭代 0 加完才能開始

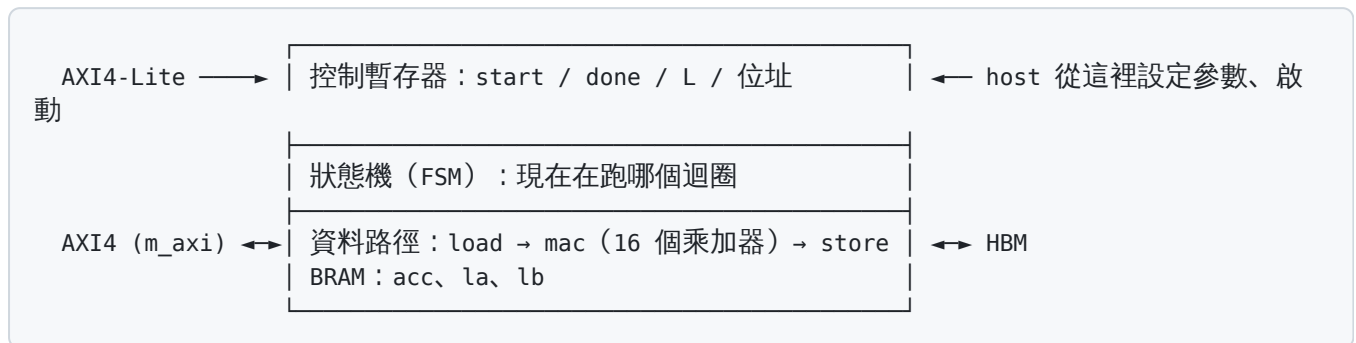
假設浮點加法需要 4 個 cycle，II 至少是 4。這叫 loop-carried dependency (跨迭代的相依)。v1~v4 把迴圈順序換掉，讓相鄰的迭代更新不同的 acc 位置，就沒有這個問題 (再用 DEPENDENCE pragma 告訴 HLS)。

④ 配置與綁定：用幾個硬體？

- 一個 pipeline 迴圈裡如果每個 cycle 都要做 16 次乘法，就需要 16 個乘法器。
- 如果迴圈沒有 pipeline，HLS 可能讓多個運算共用同一個乘法器 (省資源，但變慢)。

- 記憶體的限制：一塊 BRAM 只有 2 個讀寫埠，一個 cycle 最多讀寫 2 次。v2 每個 cycle 要讀 16 個 la，所以必須用 ARRAY_PARTITION 把陣列切成 16 塊，每塊各自讀。

⑤ 產生 RTL：電路長什麼樣子



- #pragma HLS INTERFACE m_axi → 產生 AXI4 master 介面，kernel 可以主動去 HBM 讀寫。
- 在 Vitis 流程中，純量參數（例如 L）和指標的位址會自動經由 AXI4-Lite 控制暫存器傳入。

本專題用到的 pragma 一覽

Pragma	作用	在哪裡用	不加會怎樣
PIPELINE II=1	迴圈做成流水線，每 cycle 開始一次迭代	幾乎所有內層迴圈	迭代一個接一個做，慢很多倍
ARRAY_PARTITION dim=2 complete	把陣列的某一維拆成獨立的記憶體	v2~v4 的 acc、la、lb	一個 cycle 只能讀 2 個數，16 路平行做不到
DEPENDENCE variable=acc inter false	告訴 HLS 跨迭代沒有相依	v1~v4 的 mac	HLS 保守地拉高 II
INTERFACE m_axi port=a bundle=gmem0	a 走獨立的 AXI 連接埠	所有 kernel	多個指標可能共用同一個埠，互相搶頻寬
INLINE off	函式保留成獨立模組	v3 的 load_ab、mac	被併進主函式，可能無法平行執行
LOOP_TRIPCOUNT	告訴 HLS 迴圈大概跑幾次	以 L 為上限的迴圈	報告中的 latency 顯示為「？」

可合成的 C++ 有什麼限制

HLS 只能處理「能對應到固定硬體」的 C++：| 不能用 | 原因 | |--|--| | new / malloc（動態記憶體）| 硬體的記憶體大小必須在合成時決定 | | 遞迴 | 無法事先知道要幾層硬體 | | 系統呼叫（檔案、printf 等）| 硬體沒有作業系統 | | 大小不固定的陣列 | 同動態記憶體 |

所以 kernel 裡的陣列大小都寫成常數（TI、TJ、C），而讀檔、印結果都放在 testbench 或 host。

4.4 為什麼 C++ 可以直接模擬？

核心觀念：C++ 程式就是「功能規格」

HLS 的承諾是：產生的電路，對同樣的輸入，會算出和 C++ 程式一樣的結果（在可合成的範圍內）。所以：

1. 用一般編譯器 (g++) 執行 C++ → 得到「應該要算出來的答案」。
2. 如果這個答案是對的，那 HLS 產生的電路在功能上也是對的。

#pragma HLS 只影響硬體怎麼實作（多快、用多少資源），不影響算出什麼。g++ 不認識這些 pragma，直接忽略，所以可以照常編譯執行。

四種模擬，一層比一層接近真實

模擬	在跑什麼	驗證什麼	速度	本專題
C 模擬 (csim)	C++ 程式本身 (g++ 或 HLS 內建的 clang)	演算法對不對	秒	✓ make csim
C/RTL 協同模擬 (cosim)	HLS 產生的 Verilog，用 RTL 模擬器 (Vivado 的 xsim) 執行，testbench 仍是 C++	產生的電路和 C++ 結果一致嗎？時序正確嗎？	分鐘 ~ 小時	可選 (COSIM=1)
軟體模擬 (sw_emu)	kernel 編成一般程式，搭配 XRT 模擬	host 和 kernel 的介面對不對	快	需要 XRT
硬體模擬 (hw_emu)	kernel 的 RTL + platform 的模型 + XRT	整個系統（含記憶體介面）的行為	慢	需要 XRT

為什麼 C 模擬通過不代表一定沒問題

- C++ 沒有「時間」的概念，無法抓到時序相關的錯誤（例如 DEPENDENCE 宣告錯誤導致讀到舊值）。
- 所以正式流程中，cosim 或 hw_emu 是重要的第二道防線。

4.5 Vivado：從 RTL 到 bitstream

HLS 產生的是 Verilog（描述電路的程式碼），還不是 FPGA 能直接用的檔案。接下來由 Vivado 完成（在 Vitis 流程中由 v++ -l 自動呼叫）：

步驟	做什麼	比喻
Synthesis（邏輯合成）	把 Verilog 轉成 LUT、FF、DSP、BRAM 組成的網表 (netlist)	把設計圖拆成樂高零件清單
Optimization	刪除多餘的邏輯、合併重複的部分	省掉不需要的零件
Placement（擺放）	決定每個零件放在晶片的哪個位置	決定每塊樂高放哪
Routing（繞線）	決定零件之間的連線走哪條路	接電線
Timing analysis	檢查最慢的路徑能不能在一個 cycle 內跑完	確認流水線每一站都來得及
Bitstream generation	把以上結果寫成設定檔	產出組裝說明書

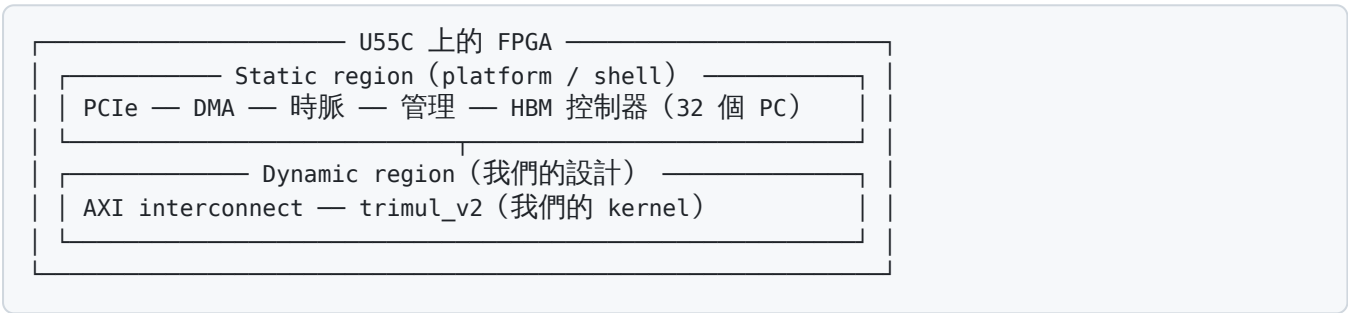
為什麼要跑 1~數小時？擺放和繞線要在數十萬到數百萬個元件中找出好的配置，這是非常難的最佳化問題，工具只能用啟發式方法反覆嘗試。

Timing 沒過會怎樣？電路在目標時脈下可能算錯，工具會自動降低時脈，或需要修改設計。HLS 報告中的 Estimated 2.431 ns 只是估計，實際值要等 Vivado 繞線完才知道。

4.6 Vitis 與 v++：把 kernel 接到板子上

Platform（平台）是什麼

FPGA 卡上除了我們的 kernel，還需要很多「基礎設施」：PCIe 介面（和主機溝通）、DMA（搬資料）、HBM 控制器、時脈產生、卡片管理。這些由 AMD 預先做好，叫做 platform 或 shell。



比喻：platform 像電腦的主機板 + 作業系統，kernel 像我們安裝的應用程式。編譯時指定的 platform 必須和卡上實際燒錄的 shell 一致，這就是為什麼 PLATFORM= 很重要。

v++ 的兩個階段

```
v++ -c (compile) : trimul_v2.cpp —(內部呼叫 HLS)—> trimul_v2.xo
                    .xo = RTL + kernel 的描述 (有哪些參數、哪些 AXI 埠)

v++ -l (link) :      trimul_v2.xo + platform + cfg —(內部呼叫 Vivado)—> trimul_v2.xclbin
                    把 kernel 放進 dynamic region、依 cfg 接到 HBM、擺放繞線、產生 bitstream
```

.xclbin 裡有什麼

- Bitstream (dynamic region 的電路)
- Metadata：kernel 名稱、每個參數的位置、每個 AXI 埠接到哪幾個 HBM PC (memory topology) → host 程式中的 krnl.group_id(0) 就是從這裡查到「參數 0 接在哪塊記憶體」。

4.7 XRT：host 程式怎麼控制 FPGA

XRT (Xilinx Runtime) 是 AMD 提供的軟體：Linux 驅動程式 + 使用者層的函式庫 + 管理工具 (xbutil)。

host 程式的動作	XRT 實際做的事
xrt::device dev(0)	透過驅動程式打開第 0 張卡

host 程式的動作	XRT 實際做的事
<code>dev.load_xclbin(...)</code>	把 bitstream 寫進 FPGA 的 dynamic region，讀取 metadata
<code>xrt::bo bo(dev, size, group)</code>	在指定的 HBM 區域配置一塊記憶體 (buffer object)
<code>bo.sync(TO_DEVICE)</code>	透過 PCIe DMA 把資料從主機記憶體複製到 HBM
<code>run.set_arg(...)</code>	把參數 (HBM 位址、L) 寫進 kernel 的 AXI4-Lite 控制暫存器
<code>run.start()</code>	寫入 start 位元，kernel 開始執行
<code>run.wait()</code>	等待 kernel 的 done 訊號
<code>bo.sync(FROM_DEVICE)</code>	把結果從 HBM 複製回主機

本專題的狀況：你們的 server63 沒有安裝 XRT、也沒有插卡，所以這部分要在有 U55C 的機器上執行。

4.8 Python 端：PyTorch、HuggingFace、ESMFold

元件	誰做的	做什麼	怎麼取得
PyTorch	Meta 主導的開源專案	張量運算、神經網路框架	<code>pip install torch</code>
transformers	HuggingFace 開源專案	各種預訓練模型的程式碼，包含 EsmForProteinFolding	<code>pip install transformers</code>
ESMFold 權重 (facebook/esmfold_v1)	Meta AI 訓練、發布在 HuggingFace Hub	模型學到的參數 (約 8 GB)	第一次執行時自動下載
蛋白質序列	UniProt、RCSB PDB (公開資料庫)	輸入資料	腳本自動從網路下載

`dump_trimul_inputs.py` 的做法是「攔截」：把 ESMFold 裡某個函式換成我們自己的版本，當模型執行到那裡時把資料存下來。模型本身完全沒有修改。

4.9 誰提供了什麼能力？

這是你特別問的問題，這裡清楚劃分。

工具鏈：都是現成的軟體，不是 Claude 做的

能力	提供者	在哪裡	怎麼來的
C++ → 電路 (HLS)	AMD Vitis HLS	你們的 server (/tools/Xilinx/2025.2/)	實驗室 / TSRI 安裝的商用軟體 (需要授權， /tools/Xilinx/flexlm 是授權管理)

能力	提供者	在哪裡	怎麼來的
RTL → bitstream	AMD Vivado	同上	同上
把 kernel 接上 platform	AMD Vitis (v++)	同上	同上
RTL 模擬 (cosim)	AMD Vivado 模擬器 (xsim)	同上	同上
控制板子	AMD XRT	需要安裝在有卡的機器上	AMD 開源 (GitHub 上可取得)
U55C platform	AMD	需要安裝在有卡的機器上	AMD 官網下載
C 模擬	GCC (g++)	任何 Linux	開源
流程自動化	GNU Make	任何 Linux	開源
蛋白質模型	Meta ESMFold + HuggingFace transformers + PyTorch	需要時用 pip 安裝、自動下載權重	開源／公開

Claude 做的事

Claude 做了	Claude 沒有做
寫所有的原始碼：5 個 kernel、testbench、host、Makefile、HLS 腳本、cfg、Python 腳本、整理報告的腳本	沒有把 C++ 轉成電路（那是 Vitis HLS 做的，在你們的 server 上執行）
寫所有的文件（包含這份手冊）	沒有執行過 Vitis/Vivado（Claude 的環境裡沒有安裝）
設計實驗（5 個版本、要比較什麼）	沒有連到你們的 server（所有 server 上的指令都是你執行、再截圖給 Claude）
在自己的雲端環境中驗證：g++ C 模擬、Python 小模型測試、假 XRT header 編譯檢查	沒有使用什麼特別下載的模型或外掛來產生程式碼
根據你的截圖推理錯誤原因並修正（例如 2025.2 路徑 bug）	

Claude 的知識從哪裡來

Claude 是一個大型語言模型，在訓練時讀過大量公開資料，包括 AMD 的官方文件、HLS 的教科書與範例、開源程式碼、學術論文等。寫程式時使用的是這些學到的知識，不是即時查詢某個資料庫。

這也意味著 Claude 可能出錯：例如一開始不知道 2025.2 已經移除 `vitis_hls`、也不知道路徑 bug，是看到你的截圖後才修正。所以每一步都需要在真實工具上驗證，這也是為什麼流程要「從快到慢」一步步確認。

4.10 Claude 在這個對話中呼叫的工具

Claude 在一個與你的 server 完全隔離的雲端 Linux 容器裡工作。以下是實際用過的工具：

工具	做了什麼	例子
Bash（在雲端容器中執行指令）	編譯、執行、測試、git 操作	<code>g++ ... tb.cpp</code> 跑 C 模擬； <code>git commit</code> 、 <code>git push</code> ； <code>pip install torch transformers</code>
Write / Edit / Read（檔案操作）	建立與修改檔案	寫 <code>trimul_v2.cpp</code> 、修改 <code>run_hls.tcl</code>
GitHub 工具	讀取 PR 的狀態	檢查 PR #1 有沒有 CI 失敗或審查留言
排程提醒（ <code>send_later</code> ）	定時提醒自己回來檢查	每小時檢查 PR、每 10 分鐘檢查手冊進度
傳送檔案（ <code>SendUserFile</code> ）	把檔案傳給你	傳送 <code>ForClaude.zip</code>

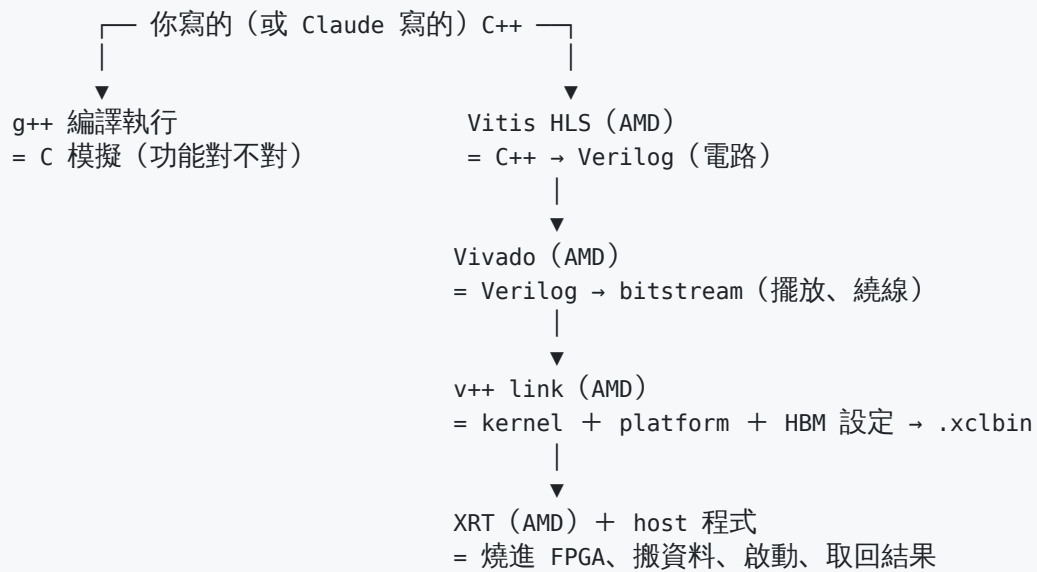
容器裡驗證了什麼、沒驗證什麼

驗證項目	方法	結果
5 個 kernel 的功能	g++ C 模擬（L = 16、32、48、64；多種 tile 大小）	✅ 全部通過
host 程式能否編譯	自己寫一組「假的」XRT header，只檢查語法	✅ 可編譯（未實際執行）
Python 擷取腳本	隨機初始化的小型 ESMFold（無法下載正式權重，容器的網路政策擋掉了 HuggingFace）	✅ 流程正確、算式差值 0
報告整理腳本	自己產生格式相同的假報告	✅ 可解析
HLS 合成	無法（容器沒有 Vitis）	由你在 server63 上執行
上板	無法	尚待找到有 U55C 的機器

安全性

- Claude 的容器無法存取你的 server，你的 server 也不會連到 Claude 的容器。
- 雙方唯一的交集是 GitHub repo：Claude 推上去，你用 `git clone` / `git pull` 下載。
- 你的 server 只有「下載」程式碼，沒有上傳任何東西。

本章小結



- 把 C++ 變成電路的能力來自 AMD 的 Vitis HLS；Claude 負責寫出「適合被轉成好電路」的 C++，以及整個實驗的設計與文件。
- C++ 能先模擬，是因為 HLS 保證電路與 C++ 的功能一致；pragma 只影響實作方式。
- 每一層工具都有對應的驗證方式，越後面的步驟越慢，所以要先把前面的驗證做好。

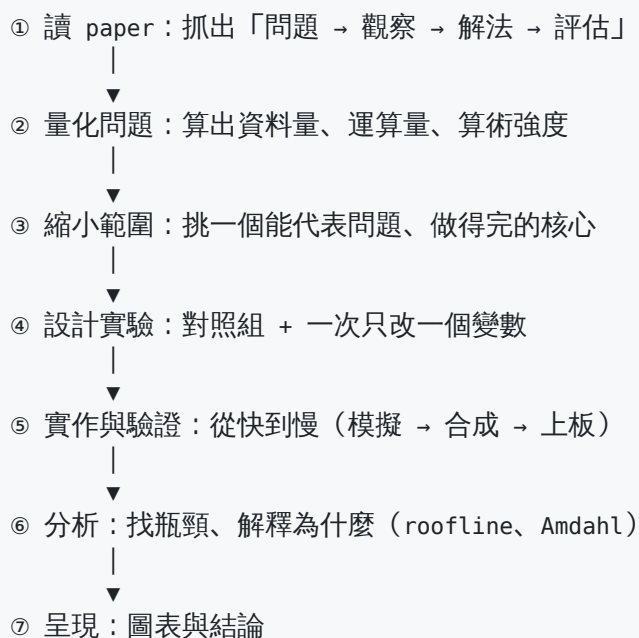
下一章整理成可以重複使用的研究方法，讓你之後讀其他 paper 時也能照這個流程進行。

第 5 章 研究方法：之後可以重複使用的步驟

本章重點

- 把這次「從一篇 paper 到可執行的實驗」的過程，整理成可以套用到任何架構／加速器 paper 的流程。
- 每一步都附上 checklist，以及本專題的實際做法當範例。
- 你之後的研究會從 LightNobel 延伸，這一章就是每次讀新 paper 時的操作手冊。

5.1 整體流程



5.2 步驟一：讀 paper 的方法

四個問題（每篇都要能回答）

問題	要找的內容	LightNobel 的答案
問題是什麼？	什麼東西不夠好？具體的限制	PPM 無法處理長序列，因為 activation 記憶體隨 L^2 成長
他們觀察到什麼？	讓解法成立的關鍵事實（通常有分析圖）	瓶頸是 activation 不是權重；不同 token 的數值分布差異很大，有 outlier
解法是什麼？	演算法 + 硬體	Token-wise 自適應量化 + 專用加速器
怎麼證明有效？	比較對象、指標、資料集	與 GPU 基準比較記憶體、速度、準確度、可支援的序列長度

閱讀順序（不要從頭讀到尾）

1. 標題、摘要：一句話說出問題與解法。
2. Introduction 的最後一兩段：通常會列出貢獻（contributions）。
3. 動機（Motivation）章節的圖：這裡是「觀察」，最重要。
4. 評估（Evaluation）的主要圖表：看比較對象和提升幅度。
5. 方法章節：前面都懂了，再看細節。
6. Related work：找出這個領域還有哪些相關論文，建立文獻地圖。

讀完要寫下的筆記模板

論文：
一句話摘要：
問題（附具體數字）：
關鍵觀察（附圖號）：
解法：
- 演算法層面：
- 硬體層面：
評估：
- 比較對象：
- 主要指標與結果：
- 資料集／模型：
我可以重現／延伸的部分：
我不確定的地方（要回原文確認）：

5.3 步驟二：量化問題

任何「效能」問題，都先算這三個數字：

數字	怎麼算	本專題的例子
資料量	每個張量的形狀 × 每個元素的 bytes	pair tensor = $L^2 \times 128 \times 4$ bytes ($L = 1000$ 時 512 MB)
運算量	數迴圈：每個輸出要幾次乘加	$2 \times L^3 \times C$ FLOP ($L = 1000$ 時約 2.6×10^{11})
算術強度	運算量 ÷ 實際搬運量	直白寫法 0.25，tiling 後 2~8 FLOP/byte

然後和硬體比：

硬體平衡點 = 峰值運算能力 ÷ 記憶體頻寬

算術強度 < 平衡點 → memory-bound → 優先減少搬運

算術強度 > 平衡點 → compute-bound → 優先增加運算單元

Checklist

- [] 找出最大的張量是哪一個，它隨什麼參數成長 (L ? L^2 ? L^3 ?)
- [] 找出運算量最大的運算

- [] 計算不同規模 ($L = 256, 1000, 4000$) 下的數字，做成表格
- [] 判斷是記憶體容量、記憶體頻寬還是運算能力的問題

5.4 步驟三：縮小範圍

好的範圍要滿足

條件	為什麼	本專題
能代表 paper 的問題	結論才有意義	Triangle Multiplication 直接處理 L^2 的 pair tensor
算式清楚、容易驗證	能確認做對了	本質是矩陣乘法，可以用 CPU 算正確答案
能和原始模型對上	證明不是自己亂編的	用 ESMFold 驗證算式差值 0
大小做得完	時間有限	只做 einsum，不做 LayerNorm/gating
能展示多種手法	才有比較	五種記憶體手法都適用

常見的縮小方式

- 只做一個運算（本專題的做法）
- 只做一種資料精度
- 固定部分參數（例如固定 $C = 128$ ，只讓 L 變動）
- 用模擬資料代替真實資料，但要說明模擬資料的特性為什麼合理（例如我們加了 1% outlier）

5.5 步驟四：設計實驗

原則一：一定要有對照組（baseline）

沒有 baseline，「快了 3 倍」就沒有意義。本專題的 v0 就是 baseline：最直白、沒有任何優化。

原則二：一次只改一個變數

```
v0 —(+ tiling)—> v1 —(+ 寬位元)—> v2 —(+ double buffering)—> v3
                        |—(+ 量化)—> v4
```

每一版只多一招，才能說「這個改善來自這一招」。v4 是從 v2 分出來的，不是從 v3。這樣 v4 和 v2 的差異就只有量化。

原則三：掃描參數，不要只看一個點

掃描	看什麼
Tile 大小 (4、8、16、32)	搬運量 vs 晶片內記憶體的 trade-off

掃描	看什麼
序列長度 L (128~2048)	時間是否符合 L^3 、記憶體是否符合 L^2
HBM 設定 (多通道 vs 單一)	頻寬的影響

原則四：同時量「好處」和「代價」

好處	代價
時間、GFLOP/s、頻寬	硬體資源 (BRAM、DSP、LUT、FF)
可支援的最大 L	精度損失 (rel_l2)
記憶體佔用	程式複雜度

Checklist

- [] 定義 baseline
- [] 每個版本只改一件事，並寫下「預期會改善什麼、為什麼」
- [] 先預測再量測 (例如：v3 預測快約 21%)
- [] 列出要掃描的參數
- [] 列出要記錄的指標 (好處 + 代價)

5.6 步驟五：實作與驗證

從快到慢

階段	時間	驗證什麼	失敗了怎麼辦
C 模擬	秒	算得對不對	修演算法
HLS 合成	分鐘	能不能變成電路、速度和資源估計	修 pragma、改寫法
C/RTL 協同模擬	分鐘~小時	電路和 C++ 一致嗎	檢查 DEPENDENCE 等宣告
硬體模擬	小時	整個系統的介面	修 host 或設定
上板	1~數小時編譯	真實速度	修時序、記憶體設定

前一步沒過，不進下一步。

驗證正確性的技巧

1. 用高精度 (double) 算參考答案。
2. 用相對誤差 (rel_l2) 而不是絕對誤差，因為數值大小不一。
3. 測試多種大小與設定：我們測了 $L = 16, 32, 48, 64$ ，以及 $8 \times 8, 16 \times 4, 32 \times 32$ 的 tile。
4. 刻意加入困難的情況：outlier 就是為了讓量化的差異顯現出來。

5. 和原始模型對照：ESMFold 的「差值 0」驗證。

5.7 步驟六：分析

找瓶頸的方法

1. 看報告中每個迴圈的 cycle 數，找出佔最多時間的部分。
2. 算出各部分的比例（例：v2 中載入佔 $141 / 671 \approx 21\%$ ）。
3. 用 Amdahl's Law 算出改善那一部分的上限。
4. 在 roofline 圖上標出位置，判斷是 memory-bound 還是 compute-bound。

解釋結果時要回答

- 結果和預測一致嗎？不一致的話，為什麼？
 - 瓶頸轉移到哪裡了？
 - 付出的代價（資源、精度）值得嗎？
 - 在真實情況下（更大的 L、真實資料）結論還成立嗎？
-

5.8 步驟七：呈現（報告要畫的圖）

圖	x 軸	y 軸	要傳達的訊息
動機圖	L	記憶體 (GB)	記憶體隨 L^2 爆炸 (問題的嚴重性)
各版本比較	v0 ~ v4	時間 (對數軸) 或加速倍數	每一招的效果
Roofline	算術強度 (對數)	GFLOP/s (對數)	每一招讓 kernel 往哪裡移動
時間拆解	版本	cycle (堆疊長條：載入／計算／其他)	瓶頸在哪、怎麼轉移
Tile sweep	tile 大小	時間 + BRAM 用量 (雙 y 軸)	trade-off
量化精度	量化方式	rel_l2	per-token vs per-tensor
資源用量	版本	BRAM／DSP／LUT 百分比	代價

5.9 除錯的思路（以 2025.2 路徑 bug 為例）

1. 讀完整的錯誤訊息（包括 WARNING）

↓ 「Cannot find source file ../kernels/trimul_v2.cpp」

2. 比較預期和實際

↓ 預期是 kernels/trimul_v2.cpp，實際變成 ../kernels/...

3. 提出假設並修正

↓ 假設：相對路徑的問題 → 改成絕對路徑

4. 修正無效 → 假設錯了，找新線索

↓ 日誌顯示加入時是絕對路徑，但合成時又被改寫

5. 用算的驗證新假設

↓ 專案在下兩層，但工具只往上一層 → 工具假設專案只在下一層

6. 針對根因修正

↓ 從 hls_prj 裡面建立專案 → 成功

回報問題時的好習慣（對學長姐、助教或 Claude 都適用）

- 附上完整的指令和錯誤前後的 20~30 行輸出。
- 說明預期會發生什麼。
- 說明已經試過什麼。
- 附上環境資訊（工具版本、作業系統）。

5.10 本專題套用這個流程的完整範例

步驟	本專題的做法	產出
① 讀 paper	LightNobel：長序列受限於 activation 記憶體；token-wise 量化	第 1 章
② 量化問題	pair tensor $L^2 \times 512 \text{ B}$ ； $AI = 0.25$	第 1、2 章的表格
③ 縮小範圍	只做 Triangle Multiplication 的 einsum	trimul.h
④ 設計實驗	v0 baseline + 四種手法；tile sweep；HBM 對照；per-token vs per-tensor	5 個 kernel、cfg
⑤ 實作與驗證	C 模擬全部通過 → v2 合成成功 → 其餘進行中	make csim、make hls
⑥ 分析	v2 載入佔 21% → 預測 v3 快約 21%	第 2 章 2.6 節
⑦ 呈現	待完成全部數據後製作	hls_summary.csv → 圖表

本章小結

- 讀 paper：問題 → 觀察 → 解法 → 評估，先看動機的圖。
- 量化：資料量、運算量、算術強度，和硬體平衡點比較。
- 實驗：有 baseline、一次改一件事、先預測再量測、同時記錄好處和代價。
- 驗證：從快到慢，前一步沒過不進下一步。
- 分析：找出比例、用 Amdahl 算上限、在 roofline 上看位置。

下一章是術語表，把整份手冊出現過的專有名詞整理在一起。

第 6 章 術語表

依主題分類。每個詞附上：白話解釋、正式說明、在本專題哪裡出現。忘記某個詞的意思時，回來這裡查。

6.1 蛋白質與模型

術語	白話	說明	本專題
胺基酸 (amino acid)	珠鍊上的一顆珠子	蛋白質的基本單位，共 20 種，用一個英文字母表示	序列中的每個字母
序列長度 L	珠子的數量	蛋白質中胺基酸的個數	整份手冊的核心變數
PPM	蛋白質結構預測模型	Protein structure Prediction Model，例如 AlphaFold2、ESMFold	LightNobel 的研究對象
AlphaFold2	DeepMind 的蛋白質結構預測模型	2021 年發表，使用 MSA 與 Evoformer	第 1 章
ESMFold	Meta 的蛋白質結構預測模型	用 ESM-2 語言模型代替 MSA，結構較簡單	我們驗證算式的對象
ESM-2	蛋白質的「語言模型」	像讀文字一樣讀胺基酸序列，產生特徵	ESMFold 的第一部分
MSA	多序列比對	Multiple Sequence Alignment，把相似的蛋白質序列排在一起	AlphaFold2 的輸入，ESMFold 不需要
Evoformer / Folding trunk	模型的主體	由 48 個 block 組成，反覆更新兩種 representation	第 1 章 1.2 節
Structure module	輸出座標的部分	把特徵轉成 3D 原子座標	第 1 章 1.2 節
Sequence representation	每個胺基酸的特徵	形狀 $L \times 1024$ (ESMFold)	第 1 章
Pair representation	每一對胺基酸之間的關係	形狀 $L \times L \times 128$ ，隨 L^2 成長	本專題處理的資料
Triangle Multiplication	用「第三者」推論兩者關係	$z[i][j][c] = \sum_k a[i][k][c] \cdot b[j][k][c]$ (outgoing)	本專題實作的運算
Outgoing / Incoming	兩種三角形方向	outgoing 用 (i,k)、(j,k)；incoming 用 (k,i)、(k,j)	我們做 outgoing
Triangle Attention	三角形版本的 attention	中間值隨 L^3 成長，實作上需分塊	第 1 章 1.3 節 (未實作)

術語	白話	說明	本專題
Token	一格資料	在 pair representation 中指一個 (i, j) 位置的 128 個數字	per-token 量化的單位
Channel (C)	每格的數字個數	特徵維度，ESMFold 的 pair 為 128	trimul.h 的 C = 128
LayerNorm / Linear / Gating	模型中的標準運算	正規化、線性轉換、用 sigmoid 控制資訊流	產生 a、b 的步驟（未實作）
Chunking	分批計算	一次只算一部分以降低記憶體峰值，代價是時間	第 2 章 2.9 節
HuggingFace	公開模型的平台	提供模型程式碼（transformers）與權重下載（Hub）	<code>dump_trimul_inputs.py</code>
UniProt / PDB	公開的蛋白質資料庫	UniProt 存序列、PDB 存實驗測定的結構	下載測試序列

6.2 效能與記憶體概念

術語	白話	說明	本專題
Memory-bound	等搬資料	效能受限於記憶體頻寬或延遲，而非運算能力	v0、v2 的載入部分
Compute-bound	廚師忙不過來	效能受限於運算單元數量	v2 的 mac 部分
Weights（權重）	模型學到的參數	大小固定，與 L 無關	第 1 章 1.4 節
Activation	計算中產生的中間資料	隨輸入大小（L）成長	LightNobel 要壓縮的對象
OOM	記憶體不夠	Out of Memory	長序列在 GPU 上的問題
FLOP	一次浮點運算	一次乘法或一次加法	一次乘加 = 2 FLOP
GFLOP/s	每秒十億次運算	效能單位	v2 約 7.3 GFLOP/s
頻寬（bandwidth）	每秒能搬多少	GB/s	單一 512-bit 埠 19.2 GB/s；U55C HBM 約 460 GB/s
延遲（latency）	等多久才開始有資料	從請求到第一筆資料的時間	v2 載入的 141 cycles 大多是延遲
算術強度（AI）	每搬 1 byte 做幾次運算	$\text{FLOP} \div \text{byte}$	v0 0.25、v2 2、v4 約 7.8
Roofline 模型	判斷瓶頸的圖	可達效能 = $\min(\text{運算上限}, \text{AI} \times \text{頻寬})$	第 2 章 2.3 節
Ridge point（平衡點）	roofline 的轉折點	峰值運算 $\div$ 頻寬	判斷 memory/compute-bound

術語	白話	說明	本專題
Tiling	分批搬、重複用	把計算切成小塊，讓搬進來的資料被重用多次	v1
資料重用 (data reuse)	同一個食材做多道菜	同一份資料被多次運算使用	tile 中每個 a 用 TJ 次
Burst	一次請求連續拿很多筆	只付一次延遲	連續的 128 個 channel
Double buffering (ping-pong)	兩個料理台輪流用	一個 buffer 計算時，另一個載入下一批	v3
Little's Law	要多少資料在路上	在途資料量 = 頻寬 × 延遲	第 2 章 2.5 節
Amdahl's Law	改善的上限	加速 = $1 \div [(1-p) + p/s]$	v3 最多約 1.27 倍
Memory layout	資料在記憶體中的排列	決定能否連續讀取	<code>[row][col][channel]</code>
Stride	連續兩次存取的間距	stride 大 → 無法 burst	v0 的 stride = 128
Operator fusion	運算合併	中間值不寫回記憶體	第 2 章 2.9 節 (未實作)
Recomputation	用計算換記憶體	不存中間值，需要時重算	第 2 章 2.9 節 (未實作)

6.3 量化

術語	白話	說明	本專題
量化 (quantization)	把數字壓縮成較少的 bit	例如 FP32 (4 bytes) → INT8 (1 byte)	v4
FP32 / FP16 / BF16	浮點格式	32 / 16 / 16 bit 的浮點數	kernel 用 FP32
INT8	8 bit 整數	-128 ~ 127 (對稱量化用 -127 ~ 127)	v4
Scale (s)	量化的尺	$s = \max x \div 127$, $x \approx s \times q$	每個 token 一個
對稱量化	以 0 為中心	正負範圍相同，不需要 zero point	v4 的做法
Per-tensor	整份共用一把尺	一個 scale 給整個張量	誤差約 11%
Per-token	每格一把尺	每個 token 一個 scale	誤差約 1%
Per-channel	每個 channel 一把尺	每個 c 一個 scale (本專題未實作)	第 3 章 3.4 節提到
Outlier	特別大的值	少數數值遠大於其他值	模擬資料中 1% 的 token ×30
Fake quantization	量化再還原	用浮點數模擬量化誤差	Python 腳本評估誤差

術語	白話	說明	本專題
rel_l2	整體相對誤差	$\ z - z_{\text{ref}}\ \div \ z_{\text{ref}}\ $	testbench、host
max_abs	最大單點誤差	$\max z - z_{\text{ref}} $	testbench、host
AAQ	LightNobel 的量化方法	Token-wise Adaptive Activation Quantization	第 1 章 1.6 節

6.4 FPGA 硬體

術語	白話	說明	本專題
FPGA	可以重新接線的晶片	Field-Programmable Gate Array	U55C
ASIC	專用晶片	做好就不能改，最快最省電	LightNobel 的加速器
U55C	我們的 FPGA 卡	AMD Alveo U55C，晶片 xcu55c-fsvh2892-2L-e	整個專題
Alveo	AMD 資料中心 FPGA 卡系列	插在伺服器的 PCIe 插槽	U55C 屬於此系列
LUT	小真值表	6 輸入，可實現任意邏輯	報告的 LUT 用量
FF (Flip-Flop)	1 bit 暫存器	每個時脈更新	報告的 FF 用量
DSP	硬體乘法器	DSP48E2，27×18 bit 乘法 + 累加	浮點乘法用
BRAM	小塊晶片內記憶體	36 Kb（可拆成兩個 18 Kb），2 個讀寫埠	acc、la、lb
URAM	大塊晶片內記憶體	288 Kb	大 tile 時可能用到
HBM	高頻寬記憶體	High Bandwidth Memory，堆疊在晶片旁	U55C 有 16 GB
Pseudo-channel (PC)	HBM 的一條通道	U55C 有 32 個，每個 512 MB	cfg 中的 HBM[0:7]
PCIe	主機和卡之間的連線	資料從 CPU 記憶體搬到 HBM 走這裡	XRT 的 sync
時脈 (clock)	電路的節拍	300 MHz = 每 3.33 ns 一拍	<code>create_clock -period 3.33</code>
Cycle	一拍	一個時脈週期	報告的 latency 單位
Bitstream	FPGA 的設定檔	決定每個元件和連線的設定	包在 .xclbin 中
廠商代號 10ee	Xilinx 的 PCI ID	<code>lspci -d 10ee</code> ：可列出 Xilinx 裝置	確認 server63 沒有卡

6.5 HLS 與硬體設計

術語	白話	說明	本專題
HLS	C++ 轉電路	High-Level Synthesis	Vitis HLS
RTL	電路的描述	Register Transfer Level，用 Verilog/VHDL 撰寫	HLS 的輸出
Verilog / VHDL	硬體描述語言	描述電路結構與行為	我們不需要手寫
Kernel	在 FPGA 上跑的函式	加速的核心運算	trimul_v0 ~ v4
Pragma	給 HLS 的指示	#pragma HLS ...，不影響功能、只影響實作	第 4 章 4.3 節
Pipeline	流水線	多個迭代重疊執行	幾乎所有內層迴圈
II (Initiation Interval)	流水線的節奏	連續兩次迭代開始的間隔，1 最好	mac 迴圈 II = 1
Iteration latency	一次迭代的長度	一次迭代從頭到尾的 cycle 數	loop_k 為 671
Pipeline depth	流水線的長度	一次迭代需要走過的級數	mac 約 14
Loop-carried dependency	下一輪要等這一輪	跨迭代的資料相依，會拉高 II	v0 的 acc
Unroll (展開)	把迴圈複製成多份硬體	平行執行	v2 的 16 lane
Array partition	把陣列拆開	讓多個元素能同時存取	ARRAY_PARTITION dim=2 complete
Scheduling (排程)	決定何時做	把運算分配到 cycle	第 4 章 4.3 節
Binding (綁定)	決定用誰做	把運算分配到硬體單元	第 4 章 4.3 節
FSM	狀態機	控制電路目前在做哪一步	HLS 自動產生
AXI4 / m_axi	讀寫記憶體介面	kernel 主動讀寫 HBM 的標準介面	INTERFACE m_axi
AXI4-Lite / s_axilite	設定參數的介面	host 寫入參數和 start 訊號	L、位址
Bundle	AXI 埠的名稱	相同 bundle 共用一個埠	gmem0、gmem1、gmem2
Trip count	迴圈次數	告訴 HLS 以便估計 latency	L_TRIP = 256
Timing (時序)	電路能否在一拍內完成	Estimated 需小於 Target	2.431 ns < 3.33 ns
Synthesis (合成)	把設計轉成元件	HLS 合成 (C++→RTL) 或邏輯合成 (RTL→netlist)	兩種都有

術語	白話	說明	本專題
Place & Route	擺放與繞線	決定元件位置與連線	Vivado，耗時最久
Netlist	元件清單 + 連線	邏輯合成的輸出	Vivado 內部

6.6 AMD 工具鏈

術語	白話	說明	本專題
Vitis	AMD 的加速應用開發環境	包含 HLS、 <code>v++</code> 、 <code>vitis-run</code> 等	2025.2
Vitis HLS	C++ → RTL 工具	2024.2 起以 <code>vitis-run --mode hls</code> 執行	<code>make hls</code>
Vivado	RTL → bitstream 工具	合成、擺放、繞線、產生 bitstream	<code>v++ -l</code> 內部呼叫
<code>v++</code>	Vitis 的編譯器	<code>-c</code> 產生 <code>.xo</code> ； <code>-l</code> 產生 <code>.xclbin</code>	<code>make xclbin</code>
<code>vitis-run</code>	執行 Vitis 工具的指令	<code>--mode hls --tcl</code> 執行 HLS 腳本	<code>make hls</code>
<code>.xo</code>	編譯好的 kernel	RTL + kernel 描述	<code>v++ -c</code> 輸出
<code>.xclbin</code>	可以燒進 FPGA 的檔案	bitstream + metadata	<code>v++ -l</code> 輸出
Platform / Shell	卡上的基礎設施	PCIe、DMA、HBM 控制器等，由 AMD 提供	<code>xilinx_u55c_gen3x16_xdma_3_202210_1</code>
Static / Dynamic region	固定區／可變區	shell 在固定區，kernel 在可變區	第 4 章 4.6 節
XRT	控制卡片的軟體	Xilinx Runtime：驅動程式 + 函式庫	<code>host.cpp</code>
<code>xbutil</code>	XRT 的管理工具	查看卡片狀態、測試	<code>xbutil examine</code>
Buffer object (bo)	卡上的一塊記憶體	XRT 在 HBM 上配置的空間	<code>xrt::bo</code>
Connectivity (.cfg)	接線設定	指定 kernel 參數接到哪些 HBM PC	<code>cfg/*.cfg</code>
<code>sw_emu</code> / <code>hw_emu</code> / <code>hw</code>	三種執行目標	軟體模擬／硬體模擬／實際上板	<code>TARGET=</code>
<code>csim</code> / <code>cosim</code>	兩種 HLS 模擬	C 模擬／C 與 RTL 協同模擬	第 4 章 4.4 節
<code>xsim</code>	Vivado 的 RTL 模擬器	<code>cosim</code> 與 <code>hw_emu</code> 使用	未使用

術語	白話	說明	本專題
csynth.rpt / csynth.xml	HLS 報告	文字版／機器可讀版	<code>hls_summary.py</code> 讀 xml
Tcl	工具的腳本語言	AMD 工具用 Tcl 撰寫自動化流程	<code>run_hls.tcl</code>
flexlm	授權管理	商用軟體的授權伺服器	<code>/tools/Xilinx/flexlm</code>

6.7 軟體與開發工具

術語	白話	說明	本專題
g++	C++ 編譯器	GNU 的開源編譯器	<code>make csim</code>
clang / LLVM	另一套編譯器	Vitis HLS 的前端使用 clang	HLS 日誌
Make / Makefile	指令捷徑	把多個步驟寫成規則	<code>fpga/trimul/Makefile</code>
Testbench	測試程式	產生輸入、呼叫 kernel、檢查輸出	<code>tb/tb.cpp</code>
Host 程式	CPU 端的控制程式	準備資料、控制 FPGA	<code>host/host.cpp</code>
git clone / pull / push	下載／更新／上傳	Git 版本控制指令	取得程式碼
分支 (branch)	同一專案的不同版本	修改先放在分支，再合併進 main	<code>claude/lightnobel-...</code>
PR (Pull Request)	合併請求	請求把分支合併進 main	PR #1
Personal Access Token	GitHub 的鑰匙	代替密碼的存取權杖	clone private repo 時需要
tmux	不會斷線的終端機	SSH 斷線後程式繼續跑	跑 <code>make hls-all</code>
PyTorch	深度學習框架	張量運算	Python 腳本
transformers	預訓練模型函式庫	HuggingFace 的開源函式庫	<code>EsmForProteinFolding</code>
Colab	免費的雲端 GPU	Google 提供的筆記本環境	沒有 GPU 時的替代方案

本章小結

遇到不懂的詞，先在這裡查白話解釋，再回到「本專題」欄位指出的章節看完整說明。

第 7 章 指令手冊：做過的、可以做的、格式與用途

本章重點

- 先學會看懂指令的格式（指令、選項、參數、管線、導向），之後遇到新指令也能自己拆解。
- 依用途分成九類：Linux 基本操作、環境調查、Git、tmux、本專案的 make 指令、Vitis 工具、看結果、host 程式、Python 腳本。
- 每個指令都標示： 已經做過、 下一步會用到、 之後可以做、 小心使用。
- 本章最後有一張「目前為止實際執行過的指令」時間軸，以及常見錯誤的排查方法。

7.1 先看懂指令的格式

基本結構

指令名稱	[選項]	[參數]
↓	↓	↓
ls	-la	/opt/xilinx
git	clone -b 分支	網址
make	hls	KERNEL=trimul_v2

部分	意思	例子
指令名稱	要執行的程式	ls、git、make
短選項	一個 - 加一個字母，可以合併	-l、-a，合併成 -la
長選項	兩個 - 加單字	--help、--mode hls
參數	指令要處理的對象	檔名、資料夾、網址
子指令	有些指令底下還有功能分類	git clone、git pull 的 clone、pull

組合指令的符號

符號	意思	例子	白話
\	管線：前一個的輸出當成後一個的輸入	ls pci \ grep -i xilinx	列出所有裝置，只留下含 xilinx 的行
>	把輸出寫進檔案（覆蓋）	make summary > out.txt	結果存檔
>>	把輸出附加到檔案結尾	... \ grep ^CSV >> results.csv	累積多次結果
2>/dev/null	把錯誤訊息丟掉	ls /opt/xilinx 2>/dev/null	資料夾不存在也不顯示錯誤

符號	意思	例子	白話
<code>2>&1</code>	錯誤訊息也當成一般輸出	<code>make hls > log.txt 2>&1</code>	錯誤也一起存進檔案
<code>&&</code>	前一個成功才執行下一個	<code>git pull && make csim</code>	更新成功才測試
<code>\ </code>	前一個失敗才執行下一個	<code>make hls \ echo 失敗</code>	
<code>;</code>	不管成功與否，依序執行	<code>cd ~; ls</code>	
<code>\</code>	行尾的反斜線：指令太長時接到下一行		
<code>#</code>	後面是註解，不會執行	<code>make csim # 看到 ALL PASS</code>	手冊裡的說明文字
<code>~</code>	你的家目錄	<code>cd ~</code>	<code>/home/anthony</code>
<code>.</code> / <code>..</code>	目前資料夾 / 上一層	<code>cd ..</code>	
<code>*</code>	萬用字元	<code>ls kernels/*.cpp</code>	所有 .cpp 檔
<code>\$變數</code>	取出變數的值	<code>echo \$XILINX_XRT</code>	

變數的兩種用法

```

KERNEL=trimul_v2 vitis-run ...    # ① 寫在指令前面：只對這一個指令有效（環境變數）
make hls KERNEL=trimul_v2        # ② 寫在 make 後面：傳給 Makefile 的變數

```

常用按鍵

按鍵	用途
Tab	自動補完檔名或指令（按兩下列出所有可能）
↑ / ↓	叫出之前輸入過的指令
Ctrl + C	中斷正在執行的程式（跑錯了、卡住了都用這個）
Ctrl + R	搜尋歷史指令
Ctrl + L	清除畫面
q	離開 less、man 這類翻頁畫面

忘記怎麼用時

```

指令 --help    # 大部分指令都有簡短說明
man 指令       # 完整手冊（按 q 離開）
make help      # 本專案的指令一覽

```

7.2 Linux 基本操作

指令	格式	用途	例子	狀態
ssh	ssh 帳號@主機	登入遠端 server	ssh anthony@scd-lab-server63	✓
pwd	pwd	顯示目前所在的資料夾		☐
ls	ls [-la] [路徑]	列出檔案；-l 詳細、-a 含隱藏檔	ls /tools/Xilinx	✓
cd	cd 路徑	切換資料夾	cd ForClaude/fpga/trimul	✓
cat	cat 檔案	印出整個檔案	cat cfg/trimul_v2.cfg	☐
head	head -n 行數 檔案	印出前幾行	head -n 60 xxx.rpt	✓
tail	tail -n 行數 檔案	印出最後幾行（看錯誤訊息最常用）	tail -n 30 hls_trimul_v4.log	→ SOON
tail -f	tail -f 檔案	持續顯示檔案新增的內容（看進度）	tail -f hls_trimul_v3.log	☐
less	less 檔案	翻頁閱讀長檔案（↑↓ 翻、/ 搜尋、q 離開）	less xxx_csynth.rpt	☐
grep	grep [-i] 關鍵字 檔案	找出含關鍵字的行；-i 不分大小寫；-n 顯示行號	grep -n ERROR hls_trimul_v4.log	✓
find	find 路徑 -name "樣式"	找檔案	find /tools -iname "*u55c*"	✓
which	which 指令	這個指令裝在哪裡（找不到代表沒裝或沒載入）	which v++	✓
echo	echo 文字或\$變數	印出文字或變數	echo \$PATH	☐
mkdir	mkdir -p 資料夾	建立資料夾	mkdir -p ~/results	☐
cp / mv	cp 來源 目的	複製 / 移動（或改名）	cp hls_summary.csv ~/results/	☐
rm	rm [-r] 檔案	⚠ 刪除，無法復原；-r 刪資料夾	rm -f host.exe	⚠
df -h	df -h	硬碟剩多少空間		☐
du -sh	du -sh 資料夾	某個資料夾佔多少空間（hls_prj/ 會越來越大）	du -sh hls_prj	☐
top / htop	top	看 CPU、記憶體用量與正在跑的程式（q 離開）	確認合成是否還在跑	☐
history	history	列出輸入過的指令	history \ grep make	☐

指令	格式	用途	例子	狀態
<code>source</code>	<code>source</code> 檔案	在目前的終端機執行設定檔（載入環境變數）	見 7.3	

⚠ `rm -rf` 會直接刪除整個資料夾，不會經過資源回收筒。刪除前先用 `ls` 確認內容；想清掉本專案產生的檔案，改用 `make clean` 比較安全。

7.3 環境調查與設定

指令	用途	結果 / 狀態
<code>lspci \ grep -i -E "xilinx\ amd\ altera"</code>	列出 PCIe 裝置，找 FPGA 卡	✅ 空的
<code>lspci -d 10ee:</code>	只列 Xilinx（廠商代號 10ee）的裝置	✅ 空的 → server63 沒有卡
<code>which vivado vitis v++ vitis-run</code>	確認工具有沒有裝	✅ 都在 <code>/tools/Xilinx/2025.2/</code>
<code>which vitis_hls</code>		✅ 找不到（2025.2 已移除）
<code>ls /tools/Xilinx</code>	看裝了哪些版本	✅ 2025.2
<code>xbutil examine</code>	列出 FPGA 卡、shell、狀態（需要 XRT）	✅ command not found
<code>ls /opt/xilinx/platforms</code>	找 U55C platform	✅ 沒有
<code>find /opt /tools -maxdepth 4 -iname "*u55c*"</code>	找 U55C 相關檔案	✅ 沒有
<code>module avail</code>	有些 server 用 module 管理工具版本	✅ 沒有相關結果
<code>nvidia-smi</code>	看有沒有 NVIDIA GPU	✅ 沒有 GPU
<code>find /tools/Xilinx/2025.2 -maxdepth 6 -type d -iname "virtexuplus*"</code>	確認 U55C 晶片型號資料有裝	❌ (HLS 報告顯示 <code>virtexuplusHBM</code> ，代表有裝)

載入環境（找到有 U55C 的機器後會用到）

```
source /tools/Xilinx/2025.2/Vitis/settings64.sh # 載入 Vitis (server63 看起來已自動載入)
source /opt/xilinx/xrt/setup.sh               # 載入 XRT (有卡的機器才有)
```

- 每次登入都要執行，或寫進 `~/.bashrc` 讓它自動執行。
- 載入後 `echo $XILINX_XRT` 會顯示路徑，`make host` 需要這個變數。

在有 U55C 的機器上要跑的指令 ()

```
lspci -d 10ee:          # 應該看到兩行 (U55C 有兩個 PCIe function)
xbutil examine          # 列出卡片與 BDF (例如 0000:3b:00.1)
xbutil examine -d <BDF> # 看這張卡目前的 shell (platform) 名稱
xbutil validate -d <BDF> # 板子自我檢測 (約數分鐘)
ls /opt/xilinx/platforms # platform 資料夾名稱 = Makefile 的 PLATFORM
```

7.4 Git : 取得與更新程式碼

指令	格式	用途	狀態
<code>git clone</code>	<code>git clone -b 分支 網址</code>	第一次下載整個專案	✓
<code>git pull</code>	<code>git pull</code>	下載 Claude 推上去的最新修改 (最常用)	✓
<code>git status</code>	<code>git status</code>	看有沒有被你改過的檔案	□
<code>git log --oneline</code>	<code>git log --oneline \ head</code>	看最近的修改紀錄	□
<code>git diff</code>	<code>git diff [檔案]</code>	看你改了哪些地方	□
<code>git branch</code>	<code>git branch</code>	看目前在哪個分支	□
<code>git checkout -- 檔案</code>		⚠ 放棄你對某個檔案的修改, 還原成最新版	⚠

本專案的 clone 指令 :

```
git clone -b claude/lightnobel-memory-optimization-yhokyc
https://github.com/antony3911/ForClaude.git
```

git pull 失敗時

如果出現 `Your local changes would be overwritten` , 代表你在 server 上改過某個檔案 :

```
git status          # 看是哪個檔案
git stash           # 先把你的修改暫存起來
git pull            # 再更新
git stash pop        # 需要的話把修改拿回來 (可能要手動合併)
```

⚠ 不要在 server 上執行 **git push** 。程式碼的修改由 Claude 推上 GitHub , server 只負責下載與執行。

7.5 tmux : 斷線也不會中斷的終端機

長時間的工作 (`make hls-all` 、上板編譯) 一定要在 tmux 裡跑。

目標	做什麼	需要	時間	產出	狀態
<code>make hls-all</code>	依序合成 v0~v4，最後自動 <code>make summary</code>	Vitis	數十分鐘以上	<code>hls_<kernel>.log</code> 、 <code>hls_summary.csv</code>	● 進行中
<code>make summary</code>	讀取所有報告，印出比較表	Python 3	秒	<code>hls_summary.csv</code>	➡ soon
<code>make hls-sweep</code>	v2 的 tile 大小掃描 (4/8/16/32)，各自存成 <code>hls_prj/trimul_v2_t<N></code>	Vitis	數十分鐘	4 份報告	➡ soon
<code>make model</code>	分析模型：預測 cycle、資源、最大 L，並和 HLS 報告比對誤差	Python 3	秒	<code>model_dse.csv</code>	➡ soon
<code>make xo</code> <code>KERNEL=...</code> <code>TARGET=hw</code>	<code>v++ -c</code> ：把 kernel 編成 .xo	Vitis + platform	數分鐘	<code>build/hw/<tag>.xo</code>	■
<code>make xclbin</code> <code>KERNEL=...</code> <code>TARGET=hw</code>	<code>v++ -l</code> ：接上 platform 產生 bitstream	Vitis + platform	1~數小時	<code>build/hw/<tag>.xclbin</code>	■
<code>make host</code>	編譯 host 程式	XRT	秒	<code>host.exe</code>	■
<code>make run</code> ...	執行 (emulation 或上板)	XRT (上板需要卡)	視 L 而定	畫面上的時間、誤差、CSV 行	■
<code>make clean</code>	⚠ 刪除所有產生的檔案 (<code>build/</code> 、 <code>hls_prj/</code> 、 <code>log</code> 、 <code>csv</code>)	—	瞬間	—	⚠

⚠ `make clean` 會刪掉 `hls_summary.csv` 和所有 HLS 報告。清之前先把要保留的結果複製到別的地方：`mkdir -p ~/results && cp hls_summary.csv ~/results/`

可以調整的變數

變數	預設值	用途	例子
<code>KERNEL</code>	<code>trimul_v2</code>	選版本	<code>KERNEL=trimul_v4</code>

變數	預設值	用途	例子
TARGET	hw_emu	sw_emu (軟體模擬) / hw_emu (硬體模擬) / hw (上板)	TARGET=hw
PLATFORM	xilinx_u55c_gen3x16_xdma_3_202210_1	必須和卡上的 shell 一致	PLATFORM=<ls /opt/xilinx/platforms 看到的名稱 >
CFG	cfg/\$(KERNEL).cfg	HBM 接線設定	CFG=cfg/trimul_v2_onebank.cfg
DEFS	空	編譯時定義，目前用來改 tile 大小	DEFS="-DTILE_I=16 -DTILE_J=16"
TAG	同 KERNEL	輸出檔名，做不同實驗時避免覆蓋	TAG=v2_t16
RUN_L	16	make run 的序列長度	RUN_L=512
RUN_ARGS	空	傳給 host.exe 的額外參數	RUN_ARGS="3 tensor"
DATA	空	真實資料的資料夾 (設定後忽略 RUN_L)	DATA=../../data/spike
KERNELS	v0~v4	hls-all 要跑哪些版本	KERNELS="trimul_v3 trimul_v4"

常用組合（照實驗整理）

```
# — 功能驗證 —
make csim # ☒ 隨機資料, L=32
make csim DATA=../../data/hba # ☐ 真實資料 (有 GPU/Colab 產生資料後)

# — 實驗 A: 五個版本比較 (HLS 報告) —
make hls-all # ☒ 進行中
make hls-all KERNELS="trimul_v3 trimul_v4" # ☐ 只重跑失敗或新增的版本
make summary # ☒ 重新整理比較表

# — 實驗 B: tile 大小 —
make hls-sweep # ☒ 一次跑 4/8/16/32
make hls-sweep SWEEP_KERNEL=trimul_v4 TILES="8 16" # ☐ 換版本或尺寸
make model # ☒ 模型 vs 報告、設計空間探索

# — 上板 (找到有 U55C 的機器後) —
make xclbin KERNEL=trimul_v2 TARGET=hw PLATFORM=<實際名稱> # ☐ 1~數小時
make run KERNEL=trimul_v2 TARGET=hw RUN_L=512 # ☐

# — 實驗 C: HBM 單一通道對照 —
make run KERNEL=trimul_v2 CFG=config/trimul_v2_onebank.cfg TAG=trimul_v2_onebank TARGET=hw
RUN_L=256 # ☐

# — 實驗 D: 量化方式 —
make run KERNEL=trimul_v4 TARGET=hw RUN_L=512 # ☐ per-token (預設)
make run KERNEL=trimul_v4 TARGET=hw RUN_L=512 RUN_ARGS="3 tensor" # ☐ per-tensor

# — 實驗 E: 序列長度掃描 (收集成一個 CSV) —
for L in 128 256 512 1024; do
  make run KERNEL=trimul_v2 TARGET=hw RUN_L=$L | grep ^CSV >> results.csv
done # ☐
```

7.7 Vitis 工具的原始指令 (Makefile 背後在做什麼)

平常用 make 就好。這一節讓你知道 make 實際呼叫了什麼，除錯時會用到。

make 目標	實際執行的指令	說明
make csim	g++ -std=c++17 -O2 ... tb/tb.cpp kernels/*.cpp -o csim.exe && ./csim.exe 32	一般 C++ 編譯
make hls	KERNEL=trimul_v2 DEFS="" vitis-run --mode hls --tcl hls/run_hls.tcl	☒ 2025.2 的 HLS 指令
make xo	v++ -c -t hw --platform <P> -k trimul_v2 -I kernels -o build/hw/trimul_v2.xo kernels/trimul_v2.cpp	編譯 kernel
make xclbin	v++ -l -t hw --platform <P> --config config/trimul_v2.cfg -o build/hw/trimul_v2.xclbin build/hw/trimul_v2.xo	連結 + 擺放繞 線
make host	g++ ... host/host.cpp -I\$XILINX_XRT/include -L\$XILINX_XRT/lib - lxrt_coreutil -pthread -o host.exe	編譯 host

make 目標	實際執行的指令	說明
make run (emulation)	emconfigutil --platform <P> --od . 然後 XCL_EMULATION_MODE=hw_emu ./host.exe ...	產生模擬設定 並執行

v++ 常用選項

選項	意思
-c / -l	compile (產生 .xo) / link (產生 .xclbin)
-t hw\ hw_emu\ sw_emu	目標
--platform	platform 名稱
-k	kernel 函式名稱
--config	設定檔 (HBM 接線等)
--save-temps --temp_dir	保留中間檔案，除錯時可以看 Vivado 的報告

run_hls.tcl 可用的環境變數

```

KERNEL=trimul_v3 CSIM=1 vitis-run --mode hls --tcl hls/run_hls.tcl    # 合成前先在 HLS 裡跑 C
模擬
KERNEL=trimul_v3 COSIM=1 vitis-run --mode hls --tcl hls/run_hls.tcl  # 合成後再跑 C/RTL 協同
模擬 (較慢)

```

也可以透過 make 傳：make hls KERNEL=trimul_v3 COSIM=1 (make 會把命令列變數傳給子程式)。

7.8 看結果

HLS 報告

```

# 報告位置
ls hls_prj/trimul_v2/sol/syn/report/

# 前 60 行: timing 與 latency 摘要 
head -n 60 hls_prj/trimul_v2/sol/syn/report/trimul_v2_csynth.rpt

# 資源用量那一段 
grep -n -A 20 "Utilization Estimates" hls_prj/trimul_v2/sol/syn/report/trimul_v2_csynth.rpt

# 只看各迴圈的 latency 與 II 
grep -n -A 15 "\* Loop:" hls_prj/trimul_v3/sol/syn/report/trimul_v3_csynth.rpt

```

make hls-all 的進度與錯誤

```
ls hls_*.log          # 哪些版本跑過了
tail -n 30 hls_trimul_v4.log # 看某一版最後的輸出（失敗時截圖這個）
grep -n "ERROR" hls_trimul_*.log # 一次找出所有錯誤
tail -f hls_trimul_v3.log  # 即時看正在跑的版本（Ctrl+C 離開）
```



比較表

```
make summary          # 印出表格並存成 hls_summary.csv
column -s, -t < hls_summary.csv # 在終端機把 CSV 排整齊
python3 scripts/hls_summary.py --L 256 # 直接呼叫腳本（--L 要和 L_TRIP 一致）
```



把結果帶回自己的電腦

在你自己的電腦（Windows 的 PowerShell 或命令提示字元）執行：

```
scp anthony@scd-lab-server63:~/ForClaude/fpga/trimul/hls_summary.csv C:\ForClaude\
```

scp 的格式是 scp 來源 目的，遠端路徑寫成 帳號@主機:路徑。

7.9 host 程式與 Python 腳本的參數

host.exe（上板用，）

```
./host.exe <xclbin> <kernel> <L> [reps=3] [token|tensor] [samples=2000] [data_dir]
```

參數	意思
xclbin	編譯好的電路檔
kernel	kernel 名稱（trimul_v2 等）
L	序列長度，必須是 tile 大小的倍數（給了 data_dir 就以檔案為準）
reps	重複執行幾次，取最快的一次
token / tensor	v4 的量化方式
samples	隨機抽幾個點驗證正確性
data_dir	真實資料的資料夾

輸出的最後一行是 CSV：

```
CSV, kernel, L, time_ms, gflops, est_traffic_GB, est_GBps, hbm_footprint_MB, rel_l2, max_abs
```

python/dump_trimul_inputs.py (需要 GPU 或 Colab , )

```
python dump_trimul_inputs.py (--uniprot ID | --pdb ID | --fasta 檔案 | --seq 序列) --out 資料夾
                                [--block 0~47] [--max-len N] [--pad 32] [--device cuda|cpu]
                                [--chunk-size 64] [--eval-channels 16]
```

例：python dump_trimul_inputs.py --uniprot P0DTC2 --out ../data/spike

docs/manual/build_pdf.py (重新產生這份手冊的 PDF)

```
pip install markdown
python3 docs/manual/build_pdf.py    # 產生 docs/manual/manual.html 與 manual.pdf
```

7.10 目前為止實際執行過的指令 (時間軸)

#	指令	在哪裡	結果
1	lspci \ grep -i -E "xilinx\ amd\ altera\ intel.*fpga"	server63	空的
2	which vivado vitis vitis_hls v++ xbutil	server63	Vivado/Vitis/v++ 有 ; vitis_hls、xbutil 沒有
3	ls /tools/Xilinx /opt/Xilinx /opt/xilinx	server63	2025.2 DocNav flexlm xic
4	xbutil examine	server63	command not found
5	which quartus aoc aocl、ls /opt/intelFPGA*	server63	沒有 (不是 Intel FPGA)
6	module avail \ grep ...	server63	沒有結果
7	lspci -d 10ee:	server63	空的 → 沒有 FPGA 卡
8	ls /opt/xilinx/platforms、find ... "*u55c*"	server63	沒有 platform
9	which vitis-run g++ make git	server63	都有
10	git clone -b ... ForClaude.git	server63	需要 GitHub 帳號 (private repo)
11	cd ForClaude/fpga/trimul	server63	
12	make csim	server63	(待確認是否 ALL PASS)
13	make hls KERNEL=trimul_v2	server63	失敗 3 次 (2025.2 路徑問題) → 修正後成功
14	git pull	server63	取得修正版
15	head -n 60 .../trimul_v2_csynth.rpt	server63	第一筆數據 : 177,029,185 cycles
16	tmux → make hls-all	server63	 進行中

7.11 常見錯誤與排查

看到的訊息	意思	怎麼辦
<code>command not found</code>	沒裝，或環境沒載入	<code>which</code> 指令；試 <code>source .../settings64.sh</code> ；問管理員
<code>No such file or directory</code>	路徑打錯，或不在正確的資料夾	<code>pwd</code> 看自己在哪； <code>ls</code> 看檔案在不在；善用 Tab 補完
<code>Permission denied</code>	沒有權限	不要用 <code>sudo</code> 硬來，先問管理員
<code>make: *** No rule to make target</code>	不在 <code>fpga/trimul</code> 裡，或目標名稱打錯	<code>cd ~/ForClaude/fpga/trimul</code> ； <code>make help</code>
Username for 'https://github.com'	repo 是 private	改成 public，或用 fine-grained token
Your local changes would be overwritten	server 上的檔案被改過	見 7.4 節的 <code>git stash</code>
<code>ERROR: [HLS ...]</code>	HLS 失敗	<code>tail -n 30 hls_<kernel>.log</code> 截圖給 Claude
<code>WARNING: [HLS ...]</code>	通常可以忽略	只有同時出現 <code>ERROR</code> 才需要處理
<code>No space left on device</code>	硬碟滿了	<code>du -sh hls_prj build</code> ；確認結果已保存後 <code>make clean</code>
畫面卡住很久沒動	可能正常（合成很慢），也可能卡住	另開一個 tmux 視窗用 <code>top</code> 看 CPU 有沒有在跑

回報問題時請提供

1. 完整的指令
2. 錯誤前後 20~30 行的輸出（`tail -n 30 xxx.log`）
3. 你預期會發生什麼
4. 已經試過什麼

本章小結

- 指令 = 名稱 + 選項 + 參數；用 `|`、`>`、`&&` 串接；忘了就 `--help`。
- 本專案的所有操作都可以用 `make 目標 變數=值` 完成，`make -n` 可以預覽不執行。
- 現在最常用的四個指令：`git pull`、`make hls-all`、`tail -n 30 hls_*.log`、`make summary`。
- 長時間工作用 tmux；刪除前先確認，不要在 server 上 `git push`。